

12 데이터 분석 예제 & 고급 *numpy*





강의 내용

Contents of Lecture

기간	내용	과제
01주차	오리엔테이션 Python 언어의 기본, lpython, 주피터 노트북	-
02주차	내장 자료구조, 함수, 파일	실습 과제
03주차	Numpy 기본 : 배열과 벡터 연산	실습 과제
04주차	Pandas 시작하기	실습 과제
05주차	데이터 로딩과 저장, 파일 형식	실습 과제
06주차	데이터 정제 및 준비(조인, 병합, 변형)	실습 과제
07주차	그래프와 시각화	기말 프로젝트 상세 공지
08주차	중간고사 - 오프라인 오픈북으로 진행	실습 과제

기간	내용	과제
09주차	데이터 집계와 그룹 연산	실습 과제
10주차	시계열	실습 과제
11주차	고급 pandas	실습 과제
12주차	파이썬 모델링 라이브러리	실습 과제
13주차	데이터 분석 예제 / 고급 Numpy	실습 과제
14주차	Kaggle / Dacon 데이터 분석 사례 Case Study	프로젝트 레포트 제출
15주차	[기말고사] Team별 프로젝트 결과 발표	종강~!

1부: 데이터 분석 예제 & 고급 *numpy*

- | | |
|--------------------------|---------------------|
| 1) bit.ly의 1.usa.gov 데이터 | 6) ndarray 객체 구조 |
| 2) movielens 영화 평점 데이터 | 7) 고급 배열 조작 기법 |
| 3) 신생아 이름 | 8) 브로드캐스팅 |
| 4) 미국농무부 영양소 정보 | 9) 고급 ufunc |
| 5) 2012년 연방선거관리위원회 DB | 10) 구조화된 배열과 레코드 배열 |
| | 11) 정렬 알고리즘 |





Bit.ly의 1.USA.gov 데이터

1.USA.gov Data from Bitly

14.1 1.USA.gov Data from Bitly

In 2011, URL shortening service **Bitly** partnered with the US government website **USA.gov** to provide a feed of anonymous data gathered from users who shorten links ending with **.gov** or **.mil**. In 2011, a live feed as well as hourly snapshots were available as downloadable text files. This service is shut down at the time of this writing (2017), but we preserved one of the data files for the book's examples.

In the case of the hourly snapshots, each line in each file contains a common form of web data known as JSON, which stands for JavaScript Object Notation. For example, if we read just the first line of a file we may see something like this:

```
In [5]: path = 'datasets/bitly_usagov/example.txt'

In [6]: open(path).readline()
Out[6]: '{ "a": "Mozilla\\5.0 (Windows NT 6.1; WOW64) AppleWebKit\\535.11
(KHTML, like Gecko) Chrome\\17.0.963.78 Safari\\535.11", "c": "US", "nk": 1,
"tz": "America\\New_York", "gr": "MA", "g": "A6qOVH", "h": "wFLQtf", "l":
"orofrog", "al": "en-US,en;q=0.8", "hh": "1.usa.gov", "r":
"http:\\\\www.facebook.com\\l\\7AQEFzjSi\\1.usa.gov\\wFLQtf", "u":
"http:\\\\www.ncbi.nlm.nih.gov\\pubmed\\22415991", "t": 1331923247, "hc":
1331822918, "cy": "Danvers", "ll": [ 42.576698, -70.954903 ] }\\n'
```

Python has both built-in and third-party libraries for converting a JSON string into a Python dictionary object. Here we'll use the `json` module and its `loads` function invoked on each line in the sample file we downloaded:

```
import json
path = 'datasets/bitly_usagov/example.txt'
records = [json.loads(line) for line in open(path)]
```

The resulting object records is now a list of Python dicts:

```
In [18]: records[0]
Out[18]:
{'a': 'Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/535.11 (KHTML, like Gecko)
Chrome/17.0.963.78 Safari/535.11',
'al': 'en-US,en;q=0.8',
'c': 'US',
'cy': 'Danvers',
'g': 'A6qOVH',
'gr': 'MA',
'h': 'wFLQtf',
'hc': 1331822918,
'hh': '1.usa.gov',
'l': 'orofrog',
'll': [42.576698, -70.954903],
'nk': 1,
'r': 'http://www.facebook.com/l/7AQEFzjSi/1.usa.gov/wFLQtf',
't': 1331923247,
'tz': 'America/New_York',
'u': 'http://www.ncbi.nlm.nih.gov/pubmed/22415991'}
```



Bit.ly의 1.USA.gov 데이터 > 순수 파이썬으로 표준시간대 세어보기

1.USA.gov Data from Bitly

Counting Time Zones in Pure Python

Suppose we were interested in finding the most often-occurring time zones in the dataset (the `tz` field). There are many ways we could do this. First, let's extract a list of time zones again using a list comprehension:

```
In [12]: time_zones = [rec['tz'] for rec in records]
-----
KeyError                                Traceback (most recent call last)
<ipython-input-12-db4fbd348da9> in <module>()
----> 1 time_zones = [rec['tz'] for rec in records]
<ipython-input-12-db4fbd348da9> in <listcomp>(.0)
----> 1 time_zones = [rec['tz'] for rec in records]
KeyError: 'tz'
```

Oops! Turns out that not all of the records have a time zone field. This is easy to handle, as we can add the check if `'tz' in rec` at the end of the list comprehension:

```
In [13]: time_zones = [rec['tz'] for rec in records if 'tz' in rec]

In [14]: time_zones[:10]
Out[14]:
['America/New_York',
 'America/Denver',
 'America/New_York',
 'America/Sao_Paulo',
 'America/New_York',
 'America/New_York',
 'Europe/Warsaw',
 '',
 '',
 '']
```

Just looking at the first 10 time zones, we see that some of them are unknown (empty string). You can filter these out also, but I'll leave them in for now. Now, to produce counts by time zone I'll show two approaches: the harder way (using just the Python standard library) and the easier way (using pandas). One way to do the counting is to use a dict to store counts while we iterate through the time zones:

```
def get_counts(sequence):
    counts = {}
    for x in sequence:
        if x in counts:
            counts[x] += 1
        else:
            counts[x] = 1
    return counts
```

Using more advanced tools in the Python standard library, you can write the same thing more briefly:

```
from collections import defaultdict

def get_counts2(sequence):
    counts = defaultdict(int) # values will initialize to 0
    for x in sequence:
        counts[x] += 1
    return counts
```

I put this logic in a function just to make it more reusable. To use it on the time zones, just pass the `time_zones` list:

```
In [17]: counts = get_counts(time_zones)

In [18]: counts['America/New_York']
Out[18]: 1251

In [19]: len(time_zones)
Out[19]: 3440
```



Bit.ly의 1.USA.gov 데이터 > 순수 파이썬으로 표준시간대 세어보기

1.USA.gov Data from Bitly

If we wanted the top 10 time zones and their counts, we can do a bit of dictionary acrobatics:

```
def top_counts(count_dict, n=10):  
    value_key_pairs = [(count, tz) for tz, count in count_dict.items()]  
    value_key_pairs.sort()  
    return value_key_pairs[-n:]
```

We have then:

```
In [21]: top_counts(counts)  
Out[21]:  
[(33, 'America/Sao_Paulo'),  
 (35, 'Europe/Madrid'),  
 (36, 'Pacific/Honolulu'),  
 (37, 'Asia/Tokyo'),  
 (74, 'Europe/London'),  
 (191, 'America/Denver'),  
 (382, 'America/Los_Angeles'),  
 (400, 'America/Chicago'),  
 (521, ''),  
 (1251, 'America/New_York')]
```

If you search the Python standard library, you may find the `collections.Counter` class, which makes this task a lot easier:

```
In [22]: from collections import Counter
```

```
In [23]: counts = Counter(time_zones)
```

```
In [24]: counts.most_common(10)
```

```
Out[24]:  
[('America/New_York', 1251),  
 ('', 521),  
 ('America/Chicago', 400),  
 ('America/Los_Angeles', 382),  
 ('America/Denver', 191),  
 ('Europe/London', 74),  
 ('Asia/Tokyo', 37),  
 ('Pacific/Honolulu', 36),  
 ('Europe/Madrid', 35),  
 ('America/Sao_Paulo', 33)]
```



Bit.ly의 1.USA.gov 데이터 > pandas로 표준시간대 세어보기

1.USA.gov Data from Bitly

Counting Time Zones with pandas

Creating a DataFrame from the original set of records is as easy as passing the list of records to pandas.DataFrame:

```
In [25]: import pandas as pd

In [26]: frame = pd.DataFrame(records)

In [27]: frame.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3560 entries, 0 to 3559
Data columns (total 18 columns):
_heartbeat_    120 non-null float64
a              3440 non-null object
al            3094 non-null object
c             2919 non-null object
cy            2919 non-null object
g             3440 non-null object
gr            2919 non-null object
h             3440 non-null object
hc            3440 non-null float64
hh            3440 non-null object
kw            93 non-null object
l             3440 non-null object
ll            2919 non-null object
nk            3440 non-null float64
r             3440 non-null object
t             3440 non-null float64
tz            3440 non-null object
u             3440 non-null object
dtypes: float64(4), object(14)
memory usage: 500.7+ KB
```

```
In [28]: frame['tz'][:10]
Out[28]:
0    America/New_York
1    America/Denver
2    America/New_York
3    America/Sao_Paulo
4    America/New_York
5    America/New_York
6    Europe/Warsaw
7
8
9
Name: tz, dtype: object
```

The output shown for the frame is the *summary view*, shown for large DataFrame objects. We can then use the value_counts method for Series:

```
In [29]: tz_counts = frame['tz'].value_counts()

In [30]: tz_counts[:10]
Out[30]:
America/New_York    1251
                  521
America/Chicago     400
America/Los_Angeles 382
America/Denver      191
Europe/London        74
Asia/Tokyo           37
Pacific/Honolulu     36
Europe/Madrid        35
America/Sao_Paulo    33
Name: tz, dtype: int64
```



Bit.ly의 1.USA.gov 데이터 > pandas로 표준시간대 세어보기

1.USA.gov Data from Bitly

We can visualize this data using matplotlib. You can do a bit of munging to fill in a substitute value for unknown and missing time zone data in the records. We replace the missing values with the `fillna` method and use boolean array indexing for the empty strings:

```
In [31]: clean_tz = frame['tz'].fillna('Missing')
```

```
In [32]: clean_tz[clean_tz == ''] = 'Unknown'
```

```
In [33]: tz_counts = clean_tz.value_counts()
```

```
In [34]: tz_counts[:10]
```

```
Out[34]:
```

America/New_York	1251
Unknown	521
America/Chicago	400
America/Los_Angeles	382
America/Denver	191
Missing	120
Europe/London	74
Asia/Tokyo	37
Pacific/Honolulu	36
Europe/Madrid	35

```
Name: tz, dtype: int64
```

At this point, we can use the `seaborn` package to make a horizontal bar plot (see Figure 14-1 for the resulting visualization):

```
In [36]: import seaborn as sns
```

```
In [37]: subset = tz_counts[:10]
```

```
In [38]: sns.barplot(y=subset.index, x=subset.values)
```

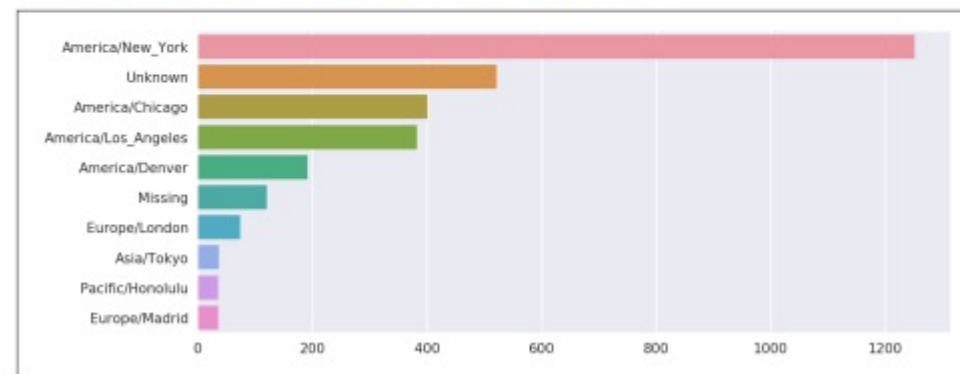


Figure 14-1. Top time zones in the 1.usa.gov sample data



Bit.ly의 1.USA.gov 데이터 > pandas로 표준시간대 세어보기

1.USA.gov Data from Bitly

The `a` field contains information about the browser, device, or application used to perform the URL shortening:

```
In [39]: frame['a'][1]
Out[39]: 'GoogleMaps/RochesterNY'

In [40]: frame['a'][50]
Out[40]: 'Mozilla/5.0 (Windows NT 5.1; rv:10.0.2) Gecko/20100101 Firefox/10.0.2'

In [41]: frame['a'][51][:50] # long line
Out[41]: 'Mozilla/5.0 (Linux; U; Android 2.2.2; en-us; LG-P9'
```

Parsing all of the interesting information in these “agent” strings may seem like a daunting task. One possible strategy is to split off the first token in the string (corresponding roughly to the browser capability) and make another summary of the user behavior:

```
In [42]: results = pd.Series([x.split()[0] for x in frame.a.dropna()])

In [43]: results[:5]
Out[43]:
0      Mozilla/5.0
1  GoogleMaps/RochesterNY
2      Mozilla/4.0
3      Mozilla/5.0
4      Mozilla/5.0
dtype: object

In [44]: results.value_counts()[:8]
Out[44]:
Mozilla/5.0      2594
Mozilla/4.0       601
GoogleMaps/RochesterNY    121
Opera/9.80        34
TEST_INTERNET_AGENT      24
GoogleProducer       21
Mozilla/6.0         5
BlackBerry8520/5.0.0.681  4
dtype: int64
```

Now, suppose you wanted to decompose the top time zones into Windows and non-Windows users. As a simplification, let's say that a user is on Windows if the string 'Windows' is in the agent string. Since some of the agents are missing, we'll exclude these from the data:

```
In [45]: cframe = frame[frame.a.notnull()]
```

We want to then compute a value for whether each row is Windows or not:

```
In [47]: cframe['os'] = np.where(cframe['a'].str.contains('Windows'),
.....:                          'Windows', 'Not Windows')

In [48]: cframe['os'][:5]
Out[48]:
0      Windows
1    Not Windows
2      Windows
3    Not Windows
4      Windows
Name: os, dtype: object
```

Then, you can group the data by its time zone column and this new list of operating systems:

```
In [49]: by_tz_os = cframe.groupby(['tz', 'os'])
```



Bit.ly의 1.USA.gov 데이터 > pandas로 표준시간대 세어보기

1.USA.gov Data from Bitly

The group counts, analogous to the value_counts function, can be computed with size. This result is then reshaped into a table with unstack:

```
In [50]: agg_counts = by_tz_os.size().unstack().fillna(0)
```

```
In [51]: agg_counts[:10]
```

```
Out[51]:
```

	Not Windows	Windows
os		
tz		
	245.0	276.0
Africa/Cairo	0.0	3.0
Africa/Casablanca	0.0	1.0
Africa/Ceuta	0.0	2.0
Africa/Johannesburg	0.0	1.0
Africa/Lusaka	0.0	1.0
America/Anchorage	4.0	1.0
America/Argentina/Buenos_Aires	1.0	0.0
America/Argentina/Cordoba	0.0	1.0
America/Argentina/Mendoza	0.0	1.0

Finally, let's select the top overall time zones. To do so, I construct an indirect index array from the row counts in agg_counts:

```
# Use to sort in ascending order
```

```
In [52]: indexer = agg_counts.sum(1).argsort()
```

```
In [53]: indexer[:10]
```

```
Out[53]:
```

tz	
	24
Africa/Cairo	20
Africa/Casablanca	21
Africa/Ceuta	92
Africa/Johannesburg	87
Africa/Lusaka	53
America/Anchorage	54
America/Argentina/Buenos_Aires	57
America/Argentina/Cordoba	26
America/Argentina/Mendoza	55

dtype: int64

I use take to select the rows in that order, then slice off the last 10 rows (largest values):

```
In [54]: count_subset = agg_counts.take(indexer[-10:])
```

```
In [55]: count_subset
```

```
Out[55]:
```

	Not Windows	Windows
os		
tz		
America/Sao_Paulo	13.0	20.0
Europe/Madrid	16.0	19.0
Pacific/Honolulu	0.0	36.0
Asia/Tokyo	2.0	35.0
Europe/London	43.0	31.0
America/Denver	132.0	59.0
America/Los_Angeles	130.0	252.0
America/Chicago	115.0	285.0
	245.0	276.0
America/New_York	339.0	912.0

pandas has a convenience method called nlargest that does the same thing:

```
In [56]: agg_counts.sum(1).nlargest(10)
```

```
Out[56]:
```

tz	
America/New_York	1251.0
	521.0
America/Chicago	400.0
America/Los_Angeles	382.0
America/Denver	191.0
Europe/London	74.0
Asia/Tokyo	37.0
Pacific/Honolulu	36.0
Europe/Madrid	35.0
America/Sao_Paulo	33.0

dtype: float64



Bit.ly의 1.USA.gov 데이터 > pandas로 표준시간대 세어보기

1.USA.gov Data from Bitly

Then, as shown in the preceding code block, this can be plotted in a bar plot; I'll make it a stacked bar plot by passing an additional argument to seaborn's barplot function (see Figure 14-2):

```
# Rearrange the data for plotting
In [58]: count_subset = count_subset.stack()

In [59]: count_subset.name = 'total'

In [60]: count_subset = count_subset.reset_index()

In [61]: count_subset[:10]
Out[61]:
```

	tz	os	total
0	America/Sao_Paulo	Not Windows	13.0
1	America/Sao_Paulo	Windows	20.0
2	Europe/Madrid	Not Windows	16.0
3	Europe/Madrid	Windows	19.0
4	Pacific/Honolulu	Not Windows	0.0
5	Pacific/Honolulu	Windows	36.0
6	Asia/Tokyo	Not Windows	2.0
7	Asia/Tokyo	Windows	35.0
8	Europe/London	Not Windows	43.0
9	Europe/London	Windows	31.0

```
In [62]: sns.barplot(x='total', y='tz', hue='os', data=count_subset)
```

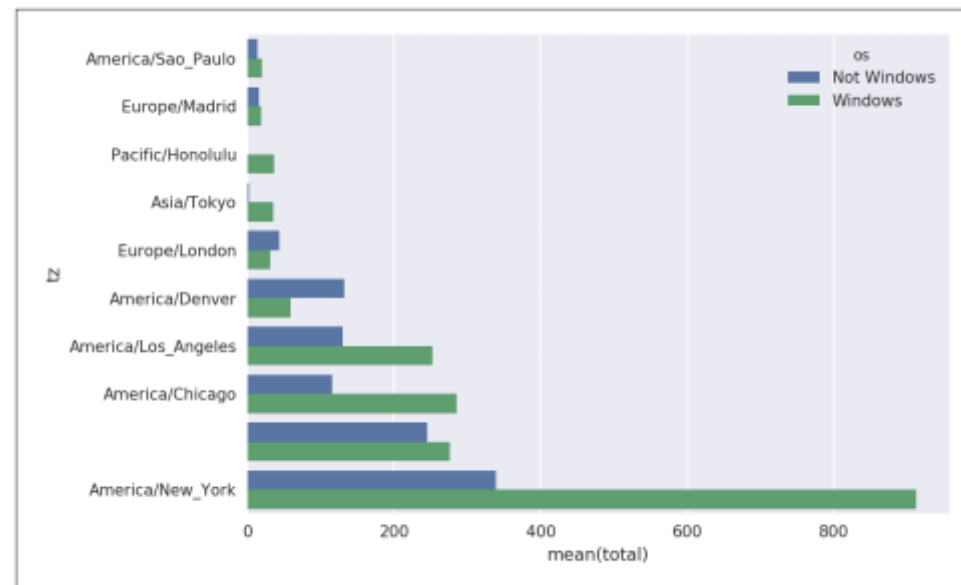


Figure 14-2. Top time zones by Windows and non-Windows users



Bit.ly의 1.USA.gov 데이터 > pandas로 표준시간대 세어보기

1.USA.gov Data from Bitly

The plot doesn't make it easy to see the relative percentage of Windows users in the smaller groups, so let's normalize the group percentages to sum to 1:

```
def norm_total(group):  
    group['normed_total'] = group.total / group.total.sum()  
    return group
```

```
results = count_subset.groupby('tz').apply(norm_total)
```

Then plot this in Figure 14-3:

```
In [65]: sns.barplot(x='normed_total', y='tz', hue='os', data=results)
```

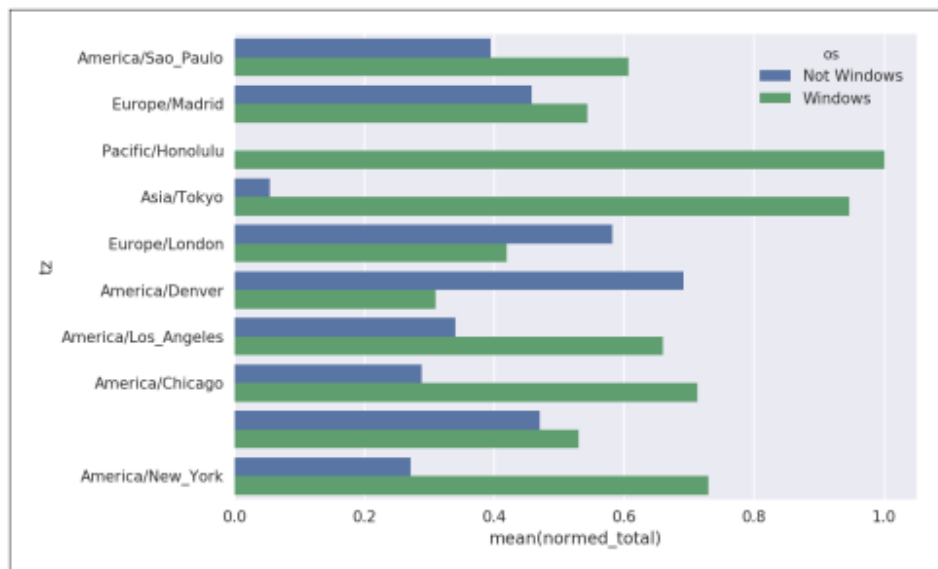


Figure 14-3. Percentage Windows and non-Windows users in top-occurring time zones

We could have computed the normalized sum more efficiently by using the transform method with groupby:

```
In [66]: g = count_subset.groupby('tz')
```

```
In [67]: results2 = count_subset.total / g.total.transform('sum')
```



MovieLens의 영화 평점 데이터

MovieLens 1M Dataset

14.2 MovieLens 1M Dataset

GroupLens Research provides a number of collections of movie ratings data collected from users of MovieLens in the late 1990s and early 2000s. The data provide movie ratings, movie metadata (genres and year), and demographic data about the users (age, zip code, gender identification, and occupation). Such data is often of interest in the development of recommendation systems based on machine learning algorithms. While we do not explore machine learning techniques in detail in this book, I will show you how to slice and dice datasets like these into the exact form you need.

The MovieLens 1M dataset contains 1 million ratings collected from 6,000 users on 4,000 movies. It's spread across three tables: ratings, user information, and movie information. After extracting the data from the ZIP file, we can load each table into a pandas DataFrame object using `pandas.read_table`:

```
import pandas as pd

# Make display smaller
pd.options.display.max_rows = 10

unames = ['user_id', 'gender', 'age', 'occupation', 'zip']
users = pd.read_table('datasets/movielens/users.dat', sep='::',
                      header=None, names=unames)

rnames = ['user_id', 'movie_id', 'rating', 'timestamp']
ratings = pd.read_table('datasets/movielens/ratings.dat', sep='::',
                        header=None, names=rnames)

mnames = ['movie_id', 'title', 'genres']
movies = pd.read_table('datasets/movielens/movies.dat', sep='::',
                       header=None, names=mnames)
```

You can verify that everything succeeded by looking at the first few rows of each DataFrame with Python's slice syntax:

```
In [69]: users[:5]
Out[69]:
   user_id gender  age  occupation  zip
0         1     F    1           10  48067
1         2     M   56           16  78072
2         3     M   25           15  55117
3         4     M   45            7  02460
4         5     M   25           20  55455

In [70]: ratings[:5]
Out[70]:
   user_id  movie_id  rating  timestamp
0         1         1     1193         5  978300760
1         1         1     661         3  978302109
2         1         1     914         3  978301968
3         1         1    3408         4  978300275
4         1         1    2355         5  978824291

In [71]: movies[:5]
Out[71]:
   movie_id  title  genres
0         1  Toy Story (1995)  Animation|Children's|Comedy
1         2  Jumanji (1995)  Adventure|Children's|Fantasy
2         3  Grumpier Old Men (1995)  Comedy|Romance
3         4  Waiting to Exhale (1995)  Comedy|Drama
4         5  Father of the Bride Part II (1995)  Comedy

In [72]: ratings
Out[72]:
   user_id  movie_id  rating  timestamp
0         1         1     1193         5  978300760
1         1         1     661         3  978302109
2         1         1     914         3  978301968
3         1         1    3408         4  978300275
4         1         1    2355         5  978824291
...      ...      ...      ...      ...
1000204    6040     1091         1  956716541
1000205    6040     1094         5  956704887
1000206    6040      562         5  956704746
1000207    6040     1096         4  956715648
1000208    6040     1097         4  956715569
[1000209 rows x 4 columns]
```



MovieLens의 영화 평점 데이터

MovieLens 1M Dataset

Note that ages and occupations are coded as integers indicating groups described in the dataset's *README* file. Analyzing the data spread across three tables is not a simple task; for example, suppose you wanted to compute mean ratings for a particular movie by sex and age. As you will see, this is much easier to do with all of the data merged together into a single table. Using pandas's `merge` function, we first merge ratings with users and then merge that result with the movies data. pandas infers which columns to use as the merge (or *join*) keys based on overlapping names:

```
In [73]: data = pd.merge(pd.merge(ratings, users), movies)
```

```
In [74]: data
```

```
Out[74]:
```

	user_id	movie_id	rating	timestamp	gender	age	occupation	zip \
0	1	1193	5	978300760	F	1	10	48067
1	2	1193	5	978298413	M	56	16	70072
2	12	1193	4	978220179	M	25	12	32793
3	15	1193	4	978199279	M	25	7	22903
4	17	1193	5	978158471	M	50	1	95350
...	...	...	...	...	...	...	...	...
1000204	5949	2198	5	958846401	M	18	17	47901
1000205	5675	2703	3	976029116	M	35	14	30030
1000206	5780	2845	1	958153068	M	18	17	92886
1000207	5851	3607	5	957756608	F	18	20	55410
1000208	5938	2909	4	957273353	M	25	1	35401
...	...	...	...	...	...	...	...	...
0							title	genres
0							One Flew Over the Cuckoo's Nest (1975)	Drama
1							One Flew Over the Cuckoo's Nest (1975)	Drama
2							One Flew Over the Cuckoo's Nest (1975)	Drama
3							One Flew Over the Cuckoo's Nest (1975)	Drama
4							One Flew Over the Cuckoo's Nest (1975)	Drama
...							...	...
1000204							Modulations (1998)	Documentary
1000205							Broken Vessels (1998)	Drama
1000206							White Boys (1999)	Drama
1000207							One Little Indian (1973)	Comedy Drama Western
1000208							Five Wives, Three Secretaries and Me (1998)	Documentary
[1000209								

rows x 10 columns]

```
In [75]: data.iloc[0]
```

```
Out[75]:
```

user_id	1
movie_id	1193
rating	5
timestamp	978300760
gender	F
age	1
occupation	10
zip	48067
title	One Flew Over the Cuckoo's Nest (1975)
genres	Drama

Name: 0, dtype: object

To get mean movie ratings for each film grouped by gender, we can use the `pivot_table` method:

```
In [76]: mean_ratings = data.pivot_table('rating', index='title',
....:                                     columns='gender', aggfunc='mean')
```

```
In [77]: mean_ratings[:5]
```

```
Out[77]:
```

gender	F	M
title		
\$1,000,000 Duck (1971)	3.375000	2.761905
'Night Mother (1986)	3.388889	3.352941
'Til There Was You (1997)	2.675676	2.733333
'burbs, The (1989)	2.793478	2.962085
...And Justice for All (1979)	3.828571	3.689024



MovieLens의 영화 평점 데이터

MovieLens 1M Dataset

This produced another DataFrame containing mean ratings with movie titles as row labels (the “index”) and gender as column labels. I first filter down to movies that received at least 250 ratings (a completely arbitrary number); to do this, I then group the data by title and use size() to get a Series of group sizes for each title:

```
In [78]: ratings_by_title = data.groupby('title').size()
```

```
In [79]: ratings_by_title[:10]
```

```
Out[79]:
```

title	
\$1,000,000 Duck (1971)	37
'Night Mother (1986)	70
'Til There Was You (1997)	52
'burbs, The (1989)	303
...And Justice for All (1979)	199
1-900 (1994)	2
10 Things I Hate About You (1999)	700
101 Dalmatians (1961)	565
101 Dalmatians (1996)	364
12 Angry Men (1957)	616

dtype: int64

```
In [80]: active_titles = ratings_by_title.index[ratings_by_title >= 250]
```

```
In [81]: active_titles
```

```
Out[81]:
```

```
Index(['burbs, The (1989)', '10 Things I Hate About You (1999)',
      '101 Dalmatians (1961)', '101 Dalmatians (1996)', '12 Angry Men (1957)',
      '13th Warrior, The (1999)', '2 Days in the Valley (1996)',
      '20,000 Leagues Under the Sea (1954)', '2001: A Space Odyssey (1968)',
      '2010 (1984)',
      ...,
      'X-Men (2000)', 'Year of Living Dangerously (1982)',
      'Yellow Submarine (1968)', 'You've Got Mail (1998)',
      'Young Frankenstein (1974)', 'Young Guns (1988)',
      'Young Guns II (1990)', 'Young Sherlock Holmes (1985)',
      'Zero Effect (1998)', 'eXistenZ (1999)'],
      dtype='object', name='title', length=1216)
```

The index of titles receiving at least 250 ratings can then be used to select rows from mean_ratings:

```
# Select rows on the index
```

```
In [82]: mean_ratings = mean_ratings.loc[active_titles]
```

```
In [83]: mean_ratings
```

```
Out[83]:
```

gender	F	M
title		
'burbs, The (1989)	2.793478	2.962085
10 Things I Hate About You (1999)	3.646552	3.311966
101 Dalmatians (1961)	3.791444	3.500000
101 Dalmatians (1996)	3.240000	2.911215
12 Angry Men (1957)	4.184397	4.328421
...	...	...
Young Guns (1988)	3.371795	3.425620
Young Guns II (1990)	2.934783	2.904025
Young Sherlock Holmes (1985)	3.514706	3.363344
Zero Effect (1998)	3.864407	3.723140
eXistenZ (1999)	3.098592	3.289086

[1216 rows x 2 columns]

To see the top films among female viewers, we can sort by the F column in descending order:

```
In [85]: top_female_ratings = mean_ratings.sort_values(by='F', ascending=False)
```

```
In [86]: top_female_ratings[:10]
```

```
Out[86]:
```

gender	F	M
title		
Close Shave, A (1995)	4.644444	4.473795
Wrong Trousers, The (1993)	4.588235	4.478261
Sunset Blvd. (a.k.a. Sunset Boulevard) (1950)	4.572650	4.464589
Wallace & Gromit: The Best of Aardman Animation...	4.563107	4.385075
Schindler's List (1993)	4.562602	4.491415
Shawshank Redemption, The (1994)	4.539075	4.560625
Grand Day Out, A (1992)	4.537879	4.293255
To Kill a Mockingbird (1962)	4.536667	4.372611
Creature Comforts (1990)	4.513889	4.272277
Usual Suspects, The (1995)	4.513317	4.518248

AI MovieLens의 영화 평점 데이터 > 평점 차이 구하기

MovieLens 1M Dataset

Measuring Rating Disagreement

Suppose you wanted to find the movies that are most divisive between male and female viewers. One way is to add a column to `mean_ratings` containing the difference in means, then sort by that:

```
In [87]: mean_ratings['diff'] = mean_ratings['M'] - mean_ratings['F']
```

Sorting by 'diff' yields the movies with the greatest rating difference so that we can see which ones were preferred by women:

```
In [88]: sorted_by_diff = mean_ratings.sort_values(by='diff')
```

```
In [89]: sorted_by_diff[:10]
```

```
Out[89]:
```

gender	F	M	diff
title			
Dirty Dancing (1987)	3.790378	2.959596	-0.830782
Jumpin' Jack Flash (1986)	3.254717	2.578358	-0.676359
Grease (1978)	3.975265	3.367041	-0.608224
Little Women (1994)	3.870588	3.321739	-0.548849
Steel Magnolias (1989)	3.901734	3.365957	-0.535777
Anastasia (1997)	3.800000	3.281609	-0.518391
Rocky Horror Picture Show, The (1975)	3.673016	3.160131	-0.512885
Color Purple, The (1985)	4.158192	3.659341	-0.498851
Age of Innocence, The (1993)	3.827068	3.339506	-0.487561
Free Willy (1993)	2.921348	2.438776	-0.482573

Reversing the order of the rows and again slicing off the top 10 rows, we get the movies preferred by men that women didn't rate as highly:

```
# Reverse order of rows, take first 10 rows
```

```
In [90]: sorted_by_diff[::-1][:10]
```

```
Out[90]:
```

gender	F	M	diff
title			
Good, The Bad and The Ugly, The (1966)	3.494949	4.221300	0.726351
Kentucky Fried Movie, The (1977)	2.878788	3.555147	0.676359
Dumb & Dumber (1994)	2.697987	3.336595	0.638608
Longest Day, The (1962)	3.411765	4.031447	0.619682
Cable Guy, The (1996)	2.250000	2.863787	0.613787
Evil Dead II (Dead By Dawn) (1987)	3.297297	3.909283	0.611985
Hidden, The (1987)	3.137931	3.745098	0.607167
Rocky III (1982)	2.361702	2.943503	0.581801
Caddyshack (1980)	3.396135	3.969737	0.573602
For a Few Dollars More (1965)	3.409091	3.953795	0.544704



MovieLens의 영화 평점 데이터 > 평점 차이 구하기

MovieLens 1M Dataset

Suppose instead you wanted the movies that elicited the most disagreement among viewers, independent of gender identification. Disagreement can be measured by the variance or standard deviation of the ratings:

```
# Standard deviation of rating grouped by title
In [91]: rating_std_by_title = data.groupby('title')['rating'].std()

# Filter down to active_titles
In [92]: rating_std_by_title = rating_std_by_title.loc[active_titles]

# Order Series by value in descending order
In [93]: rating_std_by_title.sort_values(ascending=False)[:10]
Out[93]:
title
Dumb & Dumber (1994)          1.321333
Blair Witch Project, The (1999) 1.316368
Natural Born Killers (1994)    1.307198
Tank Girl (1995)              1.277695
Rocky Horror Picture Show, The (1975) 1.260177
Eyes Wide Shut (1999)         1.259624
Evita (1996)                   1.253631
Billy Madison (1995)           1.249970
Fear and Loathing in Las Vegas (1998) 1.246408
Bicentennial Man (1999)        1.245533
Name: rating, dtype: float64
```

You may have noticed that movie genres are given as a pipe-separated (|) string. If you wanted to do some analysis by genre, more work would be required to transform the genre information into a more usable form.



14.3 US Baby Names 1880–2010

The United States Social Security Administration (SSA) has made available data on the frequency of baby names from 1880 through the present. Hadley Wickham, an author of several popular R packages, has often made use of this dataset in illustrating data manipulation in R.

We need to do some data wrangling to load this dataset, but once we do that we will have a DataFrame that looks like this:

```
In [4]: names.head(10)
Out[4]:
```

	name	sex	births	year
0	Mary	F	7065	1880
1	Anna	F	2604	1880
2	Emma	F	2003	1880
3	Elizabeth	F	1939	1880
4	Minnie	F	1746	1880
5	Margaret	F	1578	1880
6	Ida	F	1472	1880
7	Alice	F	1414	1880
8	Bertha	F	1320	1880
9	Sarah	F	1288	1880

There are many things you might want to do with the dataset:

- Visualize the proportion of babies given a particular name (your own, or another name) over time
- Determine the relative rank of a name
- Determine the most popular names in each year or the names whose popularity has advanced or declined the most
- Analyze trends in names: vowels, consonants, length, overall diversity, changes in spelling, first and last letters
- Analyze external sources of trends: biblical names, celebrities, demographic changes

With the tools in this book, many of these kinds of analyses are within reach, so I will walk you through some of them.

As of this writing, the US Social Security Administration makes available data files, one per year, containing the total number of births for each sex/name combination. The raw archive of these files can be obtained from <http://www.ssa.gov/oact/baby/names/limits.html>.

In the event that this page has been moved by the time you're reading this, it can most likely be located again by an internet search. After downloading the “National data” file *names.zip* and unzipping it, you will have a directory containing a series of files like *yob1880.txt*. I use the Unix head command to look at the first 10 lines of one of the files (on Windows, you can use the more command or open it in a text editor):

```
In [94]: !head -n 10 datasets/babynames/yob1880.txt
Mary,F,7065
Anna,F,2604
Emma,F,2003
Elizabeth,F,1939
Minnie,F,1746
Margaret,F,1578
Ida,F,1472
Alice,F,1414
Bertha,F,1320
Sarah,F,1288
```



신생아 이름

US Baby Names 1880–2010

As this is already in a nicely comma-separated form, it can be loaded into a DataFrame with `pandas.read_csv`:

```
In [95]: import pandas as pd

In [96]: names1880 = pd.read_csv('datasets/babynames/yob1880.txt',
....:                             names=['name', 'sex', 'births'])

In [97]: names1880
Out[97]:
```

	name	sex	births
0	Mary	F	7065
1	Anna	F	2604
2	Emma	F	2003
3	Elizabeth	F	1939
4	Minnie	F	1746
...	...	...	...
1995	Woodie	M	5
1996	Worthy	M	5
1997	Wright	M	5
1998	York	M	5
1999	Zachariah	M	5

[2000 rows x 3 columns]

These files only contain names with at least five occurrences in each year, so for simplicity's sake we can use the sum of the births column by sex as the total number of births in that year:

```
In [98]: names1880.groupby('sex').births.sum()
Out[98]:
```

sex	births
F	90993
M	110493

Name: births, dtype: int64

Since the dataset is split into files by year, one of the first things to do is to assemble all of the data into a single DataFrame and further to add a year field. You can do this using `pandas.concat`:

```
years = range(1880, 2011)

pieces = []
columns = ['name', 'sex', 'births']

for year in years:
    path = 'datasets/babynames/yob%d.txt' % year
    frame = pd.read_csv(path, names=columns)

    frame['year'] = year
    pieces.append(frame)

# Concatenate everything into a single DataFrame
names = pd.concat(pieces, ignore_index=True)
```

There are a couple things to note here. First, remember that `concat` glues the DataFrame objects together row-wise by default. Secondly, you have to pass `ignore_index=True` because we're not interested in preserving the original row numbers returned from `read_csv`. So we now have a very large DataFrame containing all of the names data:

```
In [100]: names
Out[100]:
```

	name	sex	births	year
0	Mary	F	7065	1880
1	Anna	F	2604	1880
2	Emma	F	2003	1880
3	Elizabeth	F	1939	1880
4	Minnie	F	1746	1880
...	...	...	...	...
1690779	Zymaire	M	5	2010
1690780	Zyonne	M	5	2010
1690781	Zyquarius	M	5	2010
1690782	Zyran	M	5	2010
1690783	Zzyzx	M	5	2010

[1690784 rows x 4 columns]



신생아 이름

US Baby Names 1880–2010

With this data in hand, we can already start aggregating the data at the year and sex level using `groupby` or `pivot_table` (see Figure 14-4):

```
In [101]: total_births = names.pivot_table('births', index='year',
.....:                                     columns='sex', aggfunc=sum)
```

```
In [102]: total_births.tail()
```

```
Out[102]:
```

sex	F	M
year		
2006	1896468	2050234
2007	1916888	2069242
2008	1883645	2032310
2009	1827643	1973359
2010	1759010	1898382

```
In [103]: total_births.plot(title='Total births by sex and year')
```

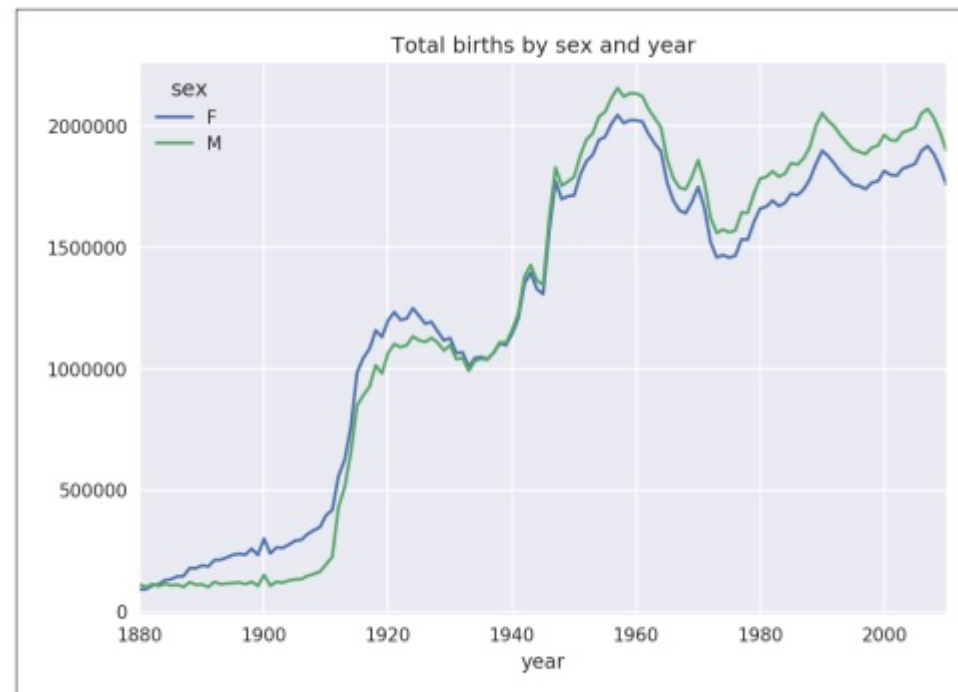


Figure 14-4. Total births by sex and year



신생아 이름

US Baby Names 1880–2010

Next, let's insert a column `prop` with the fraction of babies given each name relative to the total number of births. A `prop` value of `0.02` would indicate that 2 out of every 100 babies were given a particular name. Thus, we group the data by year and sex, then add the new column to each group:

```
def add_prop(group):
    group['prop'] = group.births / group.births.sum()
    return group
names = names.groupby(['year', 'sex']).apply(add_prop)
```

The resulting complete dataset now has the following columns:

```
In [105]: names
Out[105]:
```

	name	sex	births	year	prop
0	Mary	F	7065	1880	0.077643
1	Anna	F	2604	1880	0.028618
2	Emma	F	2003	1880	0.022013
3	Elizabeth	F	1939	1880	0.021309
4	Minnie	F	1746	1880	0.019188
...	...	..	...	...	...
1690779	Zymaire	M	5	2010	0.000003
1690780	Zyonne	M	5	2010	0.000003
1690781	Zyquarius	M	5	2010	0.000003
1690782	Zyran	M	5	2010	0.000003
1690783	Zzyzx	M	5	2010	0.000003

[1690784 rows x 5 columns]

When performing a group operation like this, it's often valuable to do a sanity check, like verifying that the `prop` column sums to 1 within all the groups:

```
In [106]: names.groupby(['year', 'sex']).prop.sum()
Out[106]:
```

year	sex	prop
1880	F	1.0
	M	1.0
1881	F	1.0
	M	1.0
1882	F	1.0
...		
2008	M	1.0
2009	F	1.0
	M	1.0
2010	F	1.0
	M	1.0

Name: prop, Length: 262, dtype: float64

Now that this is done, I'm going to extract a subset of the data to facilitate further analysis: the top 1,000 names for each sex/year combination. This is yet another group operation:

```
def get_top1000(group):
    return group.sort_values(by='births', ascending=False)[:1000]
grouped = names.groupby(['year', 'sex'])
top1000 = grouped.apply(get_top1000)
# Drop the group index, not needed
top1000.reset_index(inplace=True, drop=True)
```

If you prefer a do-it-yourself approach, try this instead:

```
pieces = []
for year, group in names.groupby(['year', 'sex']):
    pieces.append(group.sort_values(by='births', ascending=False)[:1000])
top1000 = pd.concat(pieces, ignore_index=True)
```



신생아 이름

US Baby Names 1880–2010

The resulting dataset is now quite a bit smaller:

```
In [108]: top1000
Out[108]:
```

	name	sex	births	year	prop
0	Mary	F	7065	1880	0.077643
1	Anna	F	2604	1880	0.028618
2	Emma	F	2003	1880	0.022013
3	Elizabeth	F	1939	1880	0.021309
4	Minnie	F	1746	1880	0.019188
...	...	..	...	...	...
261872	Camilo	M	194	2010	0.000102
261873	Destin	M	194	2010	0.000102
261874	Jaquan	M	194	2010	0.000102
261875	Jaydan	M	194	2010	0.000102
261876	Maxton	M	193	2010	0.000102

```
[261877 rows x 5 columns]
```

We'll use this Top 1,000 dataset in the following investigations into the data.



신생아 이름 > 이름 유행 분석

US Baby Names 1880–2010

Analyzing Naming Trends

With the full dataset and Top 1,000 dataset in hand, we can start analyzing various naming trends of interest. Splitting the Top 1,000 names into the boy and girl portions is easy to do first:

```
In [109]: boys = top1000[top1000.sex == 'M']
```

```
In [110]: girls = top1000[top1000.sex == 'F']
```

Simple time series, like the number of Johns or Marys for each year, can be plotted but require a bit of munging to be more useful. Let's form a pivot table of the total number of births by year and name:

```
In [111]: total_births = top1000.pivot_table('births', index='year',
.....:                                     columns='name',
.....:                                     aggfunc=sum)
```

Now, this can be plotted for a handful of names with DataFrame's plot method (Figure 14-5 shows the result):

```
In [112]: total_births.info()
<class 'pandas.core.frame.DataFrame'>
Int64Index: 131 entries, 1880 to 2010
Columns: 6868 entries, Aaden to Zuri
dtypes: float64(6868)
memory usage: 6.9 MB
```

```
In [113]: subset = total_births[['John', 'Harry', 'Mary', 'Marilyn']]
```

```
In [114]: subset.plot(subplots=True, figsize=(12, 10), grid=False,
.....:               title="Number of births per year")
```

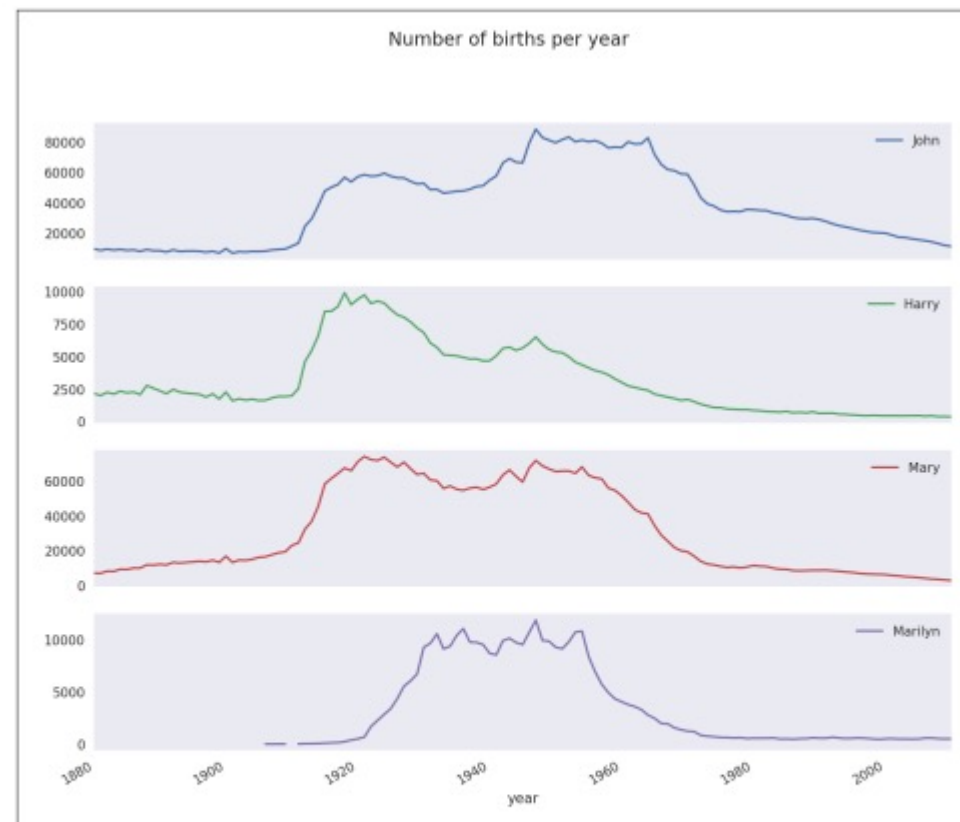


Figure 14-5. A few boy and girl names over time

On looking at this, you might conclude that these names have grown out of favor with the American population. But the story is actually more complicated than that, as will be explored in the next section.



신생아 이름 > 이름 유행 분석

US Baby Names 1880–2010

Measuring the increase in naming diversity

One explanation for the decrease in plots is that fewer parents are choosing common names for their children. This hypothesis can be explored and confirmed in the data. One measure is the proportion of births represented by the top 1,000 most popular names, which I aggregate and plot by year and sex (Figure 14-6 shows the resulting plot):

```
In [116]: table = top1000.pivot_table('prop', index='year',
.....:                                columns='sex', aggfunc=sum)

In [117]: table.plot(title='Sum of table1000.prop by year and sex',
.....:               yticks=np.linspace(0, 1.2, 13), xticks=range(1880, 2020, 10)
.....:               )
```

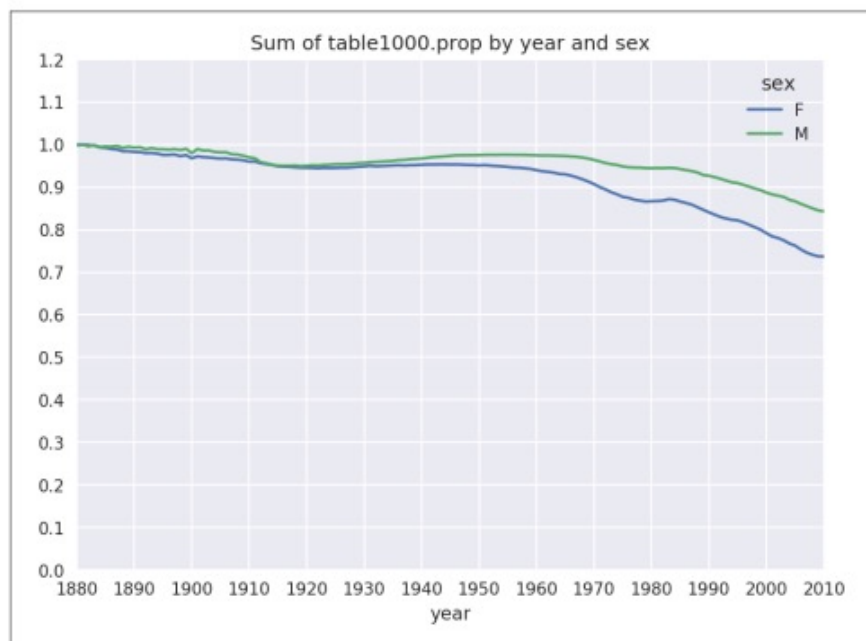


Figure 14-6. Proportion of births represented in top 1000 names by sex

You can see that, indeed, there appears to be increasing name diversity (decreasing total proportion in the top 1,000). Another interesting metric is the number of distinct names, taken in order of popularity from highest to lowest, in the top 50% of births. This number is a bit more tricky to compute. Let's consider just the boy names from 2010:

```
In [118]: df = boys[boys.year == 2010]
```

```
In [119]: df
```

```
Out[119]:
```

	name	sex	births	year	prop
260877	Jacob	M	21875	2010	0.011523
260878	Ethan	M	17866	2010	0.009411
260879	Michael	M	17133	2010	0.009025
260880	Jayden	M	17030	2010	0.008971
260881	William	M	16870	2010	0.008887
...	...	..	...	...	...
261872	Camilo	M	194	2010	0.000102
261873	Destin	M	194	2010	0.000102
261874	Jaquan	M	194	2010	0.000102
261875	Jaydan	M	194	2010	0.000102
261876	Maxton	M	193	2010	0.000102

```
[1000 rows x 5 columns]
```



신생아 이름 > 이름 유행 분석

US Baby Names 1880–2010

After sorting `prop` in descending order, we want to know how many of the most popular names it takes to reach 50%. You could write a for loop to do this, but a vectorized NumPy way is a bit more clever. Taking the cumulative sum, `cumsum`, of `prop` and then calling the method `searchsorted` returns the position in the cumulative sum at which 0.5 would need to be inserted to keep it in sorted order:

```
In [120]: prop_cumsum = df.sort_values(by='prop', ascending=False).prop.cumsum()
```

```
In [121]: prop_cumsum[:10]
```

```
Out[121]:
```

```
260877    0.011523
260878    0.020934
260879    0.029959
260880    0.038930
260881    0.047817
260882    0.056579
260883    0.065155
260884    0.073414
260885    0.081528
260886    0.089621
```

```
Name: prop, dtype: float64
```

```
In [122]: prop_cumsum.values.searchsorted(0.5)
```

```
Out[122]: 116
```

Since arrays are zero-indexed, adding 1 to this result gives you a result of 117. By contrast, in 1900 this number was much smaller:

```
In [123]: df = boys[boys.year == 1900]
```

```
In [124]: in1900 = df.sort_values(by='prop', ascending=False).prop.cumsum()
```

```
In [125]: in1900.values.searchsorted(0.5) + 1
```

```
Out[125]: 25
```

You can now apply this operation to each year/sex combination, `groupby` those fields, and apply a function returning the count for each group:

```
def get_quantile_count(group, q=0.5):
    group = group.sort_values(by='prop', ascending=False)
    return group.prop.cumsum().values.searchsorted(q) + 1
```

```
diversity = top1000.groupby(['year', 'sex']).apply(get_quantile_count)
```

```
diversity = diversity.unstack('sex')
```

This resulting DataFrame `diversity` now has two time series, one for each sex, indexed by year. This can be inspected in IPython and plotted as before (see [Figure 14-7](#)):

```
In [128]: diversity.head()
```

```
Out[128]:
```

```
sex    F    M
year
1880   38   14
1881   38   14
1882   38   15
1883   39   15
1884   39   16
```



신생아 이름 > 이름 유행 분석

US Baby Names 1880–2010

```
In [129]: diversity.plot(title="Number of popular names in top 50%")
```

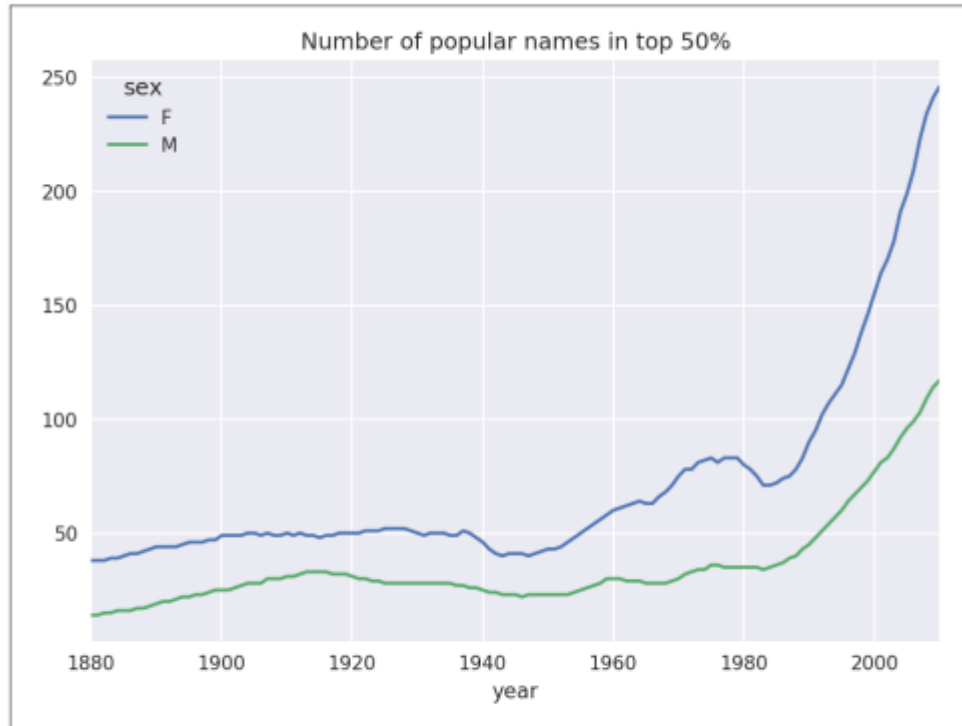


Figure 14-7. Plot of diversity metric by year

As you can see, girl names have always been more diverse than boy names, and they have only become more so over time. Further analysis of what exactly is driving the diversity, like the increase of alternative spellings, is left to the reader.



신생아 이름 > 이름 유행 분석

US Baby Names 1880–2010

The “last letter” revolution

In 2007, baby name researcher Laura Wattenberg pointed out on [her website](#) that the distribution of boy names by final letter has changed significantly over the last 100 years. To see this, we first aggregate all of the births in the full dataset by year, sex, and final letter:

```
# extract last letter from name column
get_last_letter = lambda x: x[-1]
last_letters = names.name.map(get_last_letter)
last_letters.name = 'last_letter'

table = names.pivot_table('births', index=last_letters,
                           columns=['sex', 'year'], aggfunc=sum)
```

Then we select out three representative years spanning the history and print the first few rows:

```
In [131]: subtable = table.reindex(columns=[1910, 1960, 2010], level='year')

In [132]: subtable.head()
Out[132]:
```

sex	F			M		
year	1910	1960	2010	1910	1960	2010
last_letter						
a	108376.0	691247.0	670605.0	977.0	5204.0	28438.0
b	NaN	694.0	450.0	411.0	3912.0	38859.0
c	5.0	49.0	946.0	482.0	15476.0	23125.0
d	6750.0	3729.0	2607.0	22111.0	262112.0	44398.0
e	133569.0	435013.0	313833.0	28655.0	178823.0	129012.0

Next, normalize the table by total births to compute a new table containing proportion of total births for each sex ending in each letter:

```
In [133]: subtable.sum()
Out[133]:
```

sex	year	
F	1910	396416.0
	1960	2022062.0
	2010	1759010.0
M	1910	194198.0
	1960	2132588.0
	2010	1898382.0

```
dtype: float64

In [134]: letter_prop = subtable / subtable.sum()

In [135]: letter_prop
Out[135]:
```

sex	F			M		
year	1910	1960	2010	1910	1960	2010
last_letter						
a	0.273390	0.341853	0.381240	0.005031	0.002440	0.014980
b	NaN	0.000343	0.000256	0.002116	0.001834	0.020470
c	0.000013	0.000024	0.000538	0.002482	0.007257	0.012181
d	0.017028	0.001844	0.001482	0.113858	0.122908	0.023387
e	0.336941	0.215133	0.178415	0.147556	0.083853	0.067959
...	...	...	...	...	...	...
v	NaN	0.000060	0.000117	0.000113	0.000037	0.001434
w	0.000020	0.000031	0.001182	0.006329	0.007711	0.016148
x	0.000015	0.000037	0.000727	0.003965	0.001851	0.008614
y	0.110972	0.152569	0.116828	0.077349	0.160987	0.058168
z	0.002439	0.000659	0.000704	0.000170	0.000184	0.001831

```
[26 rows x 6 columns]
```



신생아 이름 > 이름 유행 분석

US Baby Names 1880–2010

With the letter proportions now in hand, we can make bar plots for each sex broken down by year (see Figure 14-8):

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 1, figsize=(10, 8))
letter_prop['M'].plot(kind='bar', rot=0, ax=axes[0], title='Male')
letter_prop['F'].plot(kind='bar', rot=0, ax=axes[1], title='Female',
                      legend=False)
```

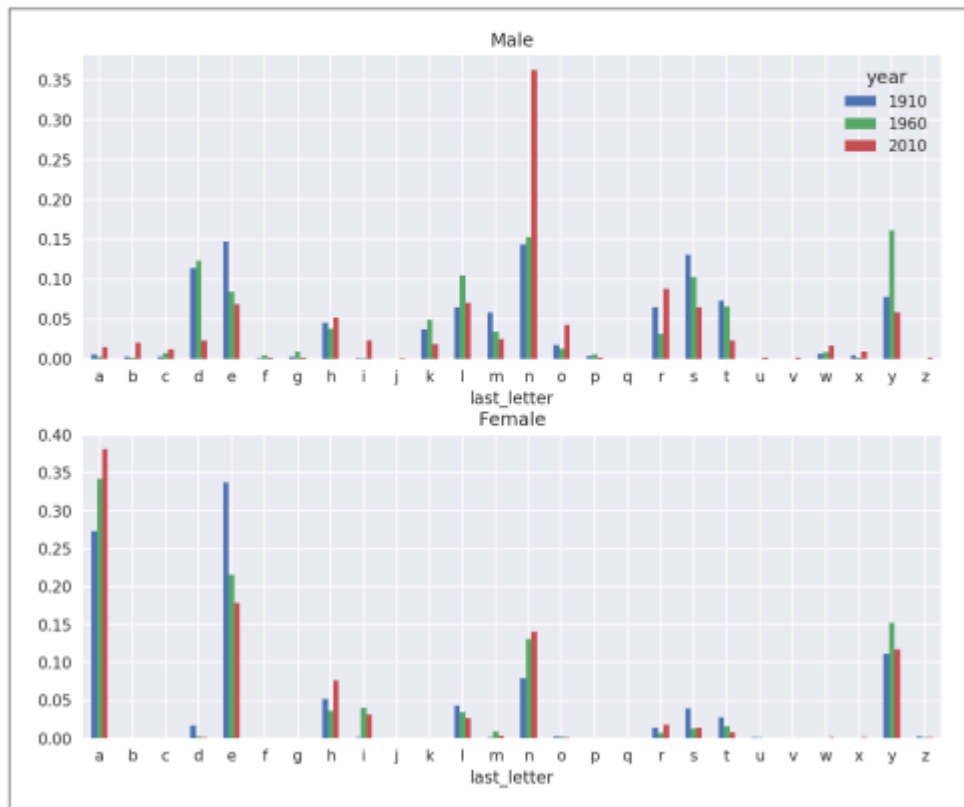


Figure 14-8. Proportion of boy and girl names ending in each letter



신생아 이름 > 이름 유행 분석

US Baby Names 1880–2010

As you can see, boy names ending in *n* have experienced significant growth since the 1960s. Going back to the full table created before, I again normalize by year and sex and select a subset of letters for the boy names, finally transposing to make each column a time series:

```
In [138]: letter_prop = table / table.sum()
```

```
In [139]: dny_ts = letter_prop.loc[['d', 'n', 'y'], 'M'].T
```

```
In [140]: dny_ts.head()
```

```
Out[140]:
```

last_letter	d	n	y
year			
1880	0.083055	0.153213	0.075760
1881	0.083247	0.153214	0.077451
1882	0.085340	0.149560	0.077537
1883	0.084066	0.151646	0.079144
1884	0.086120	0.149915	0.080405

With this DataFrame of time series in hand, I can make a plot of the trends over time again with its plot method (see [Figure 14-9](#)):

```
In [143]: dny_ts.plot()
```

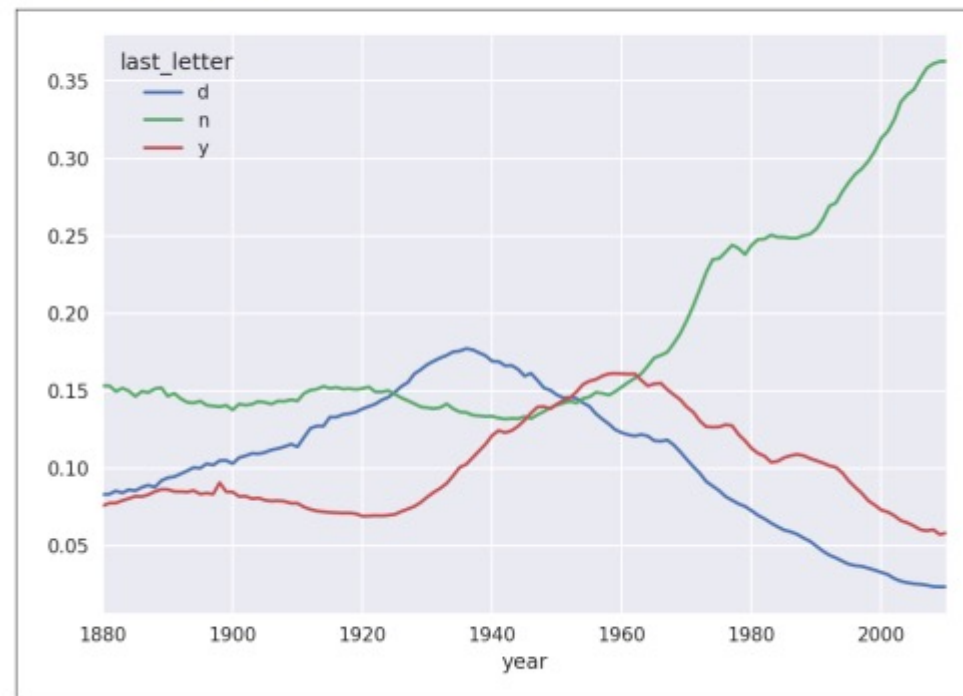


Figure 14-9. Proportion of boys born with names ending in *d/n/y* over time



신생아 이름 > 이름 유행 분석

US Baby Names 1880–2010

Boy names that became girl names (and vice versa)

Another fun trend is looking at boy names that were more popular with one sex earlier in the sample but have “changed sexes” in the present. One example is the name Lesley or Leslie. Going back to the top1000 DataFrame, I compute a list of names occurring in the dataset starting with “lesl”:

```
In [144]: all_names = pd.Series(top1000.name.unique())

In [145]: lesley_like = all_names[all_names.str.lower().str.contains('lesl')]

In [146]: lesley_like
Out[146]:
632    Leslie
2294   Lesley
4262   Leslee
4728    Lesli
6103   Lesly
dtype: object
```

From there, we can filter down to just those names and sum births grouped by name to see the relative frequencies:

```
In [147]: filtered = top1000[top1000.name.isin(lesley_like)]

In [148]: filtered.groupby('name').births.sum()
Out[148]:
name
Leslee      1082
Lesley     35022
Lesli        929
Leslie     370429
Lesly       10067
Name: births, dtype: int64
```

Next, let's aggregate by sex and year and normalize within year:

```
In [149]: table = filtered.pivot_table('births', index='year',
.....:                                  columns='sex', aggfunc='sum')

In [150]: table = table.div(table.sum(1), axis=0)

In [151]: table.tail()
Out[151]:
sex    F    M
year
2006  1.0  NaN
2007  1.0  NaN
2008  1.0  NaN
2009  1.0  NaN
2010  1.0  NaN
```

Lastly, it's now possible to make a plot of the breakdown by sex over time (Figure 14-10):

```
In [153]: table.plot(style={'M': 'k-', 'F': 'k--'})
```

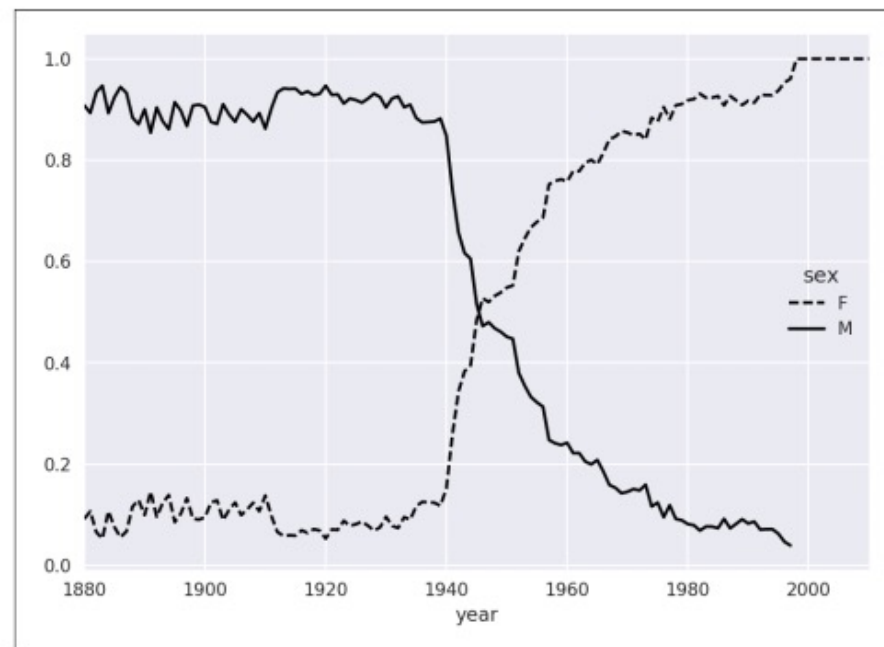


Figure 14-10. Proportion of male/female Lesley-like names over time



미국농무부 영양소 정보

USDA Food Database

14.4 USDA Food Database

The US Department of Agriculture makes available a database of food nutrient information. Programmer Ashley Williams made available a version of this database in JSON format. The records look like this:

```
{
  "id": 21441,
  "description": "KENTUCKY FRIED CHICKEN, Fried Chicken, EXTRA CRISPY,
Wing, meat and skin with breading",
  "tags": ["KFC"],
  "manufacturer": "Kentucky Fried Chicken",
  "group": "Fast Foods",
  "portions": [
    {
      "amount": 1,
      "unit": "wing, with skin",
      "grams": 68.0
    }
  ],
  ...
},
"nutrients": [
  {
    "value": 20.8,
    "units": "g",
    "description": "Protein",
    "group": "Composition"
  },
  ...
]
}
```

Each food has a number of identifying attributes along with two lists of nutrients and portion sizes. Data in this form is not particularly amenable to analysis, so we need to do some work to wrangle the data into a better form.

After downloading and extracting the data from the link, you can load it into Python with any JSON library of your choosing. I'll use the built-in Python json module:

```
In [154]: import json

In [155]: db = json.load(open('datasets/usda_food/database.json'))

In [156]: len(db)
Out[156]: 6636
```

Each entry in db is a dict containing all the data for a single food. The 'nutrients' field is a list of dicts, one for each nutrient:

```
In [157]: db[0].keys()
Out[157]: dict_keys(['id', 'description', 'tags', 'manufacturer', 'group', 'portions', 'nutrients'])
```

```
In [158]: db[0]['nutrients'][0]
Out[158]:
{'description': 'Protein',
 'group': 'Composition',
 'units': 'g',
 'value': 25.18}
```

```
In [159]: nutrients = pd.DataFrame(db[0]['nutrients'])
```

```
In [160]: nutrients[:7]
Out[160]:
```

	description	group	units	value
0	Protein	Composition	g	25.18
1	Total lipid (fat)	Composition	g	29.20
2	Carbohydrate, by difference	Composition	g	3.06
3	Ash	Other	g	3.28
4	Energy	Energy	kcal	376.00
5	Water	Composition	g	39.28
6	Energy	Energy	kJ	1573.00



미국농무부 영양소 정보

USDA Food Database

When converting a list of dicts to a DataFrame, we can specify a list of fields to extract. We'll take the food names, group, ID, and manufacturer:

```
In [161]: info_keys = ['description', 'group', 'id', 'manufacturer']
```

```
In [162]: info = pd.DataFrame(db, columns=info_keys)
```

```
In [163]: info[:5]
```

```
Out[163]:
```

	description	group	id	manufacturer
0	Cheese, caraway	Dairy and Egg Products	1008	
1	Cheese, cheddar	Dairy and Egg Products	1009	
2	Cheese, edam	Dairy and Egg Products	1018	
3	Cheese, feta	Dairy and Egg Products	1019	
4	Cheese, mozzarella, part skim milk	Dairy and Egg Products	1028	

```
In [164]: info.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 6636 entries, 0 to 6635  
Data columns (total 4 columns):  
description    6636 non-null object  
group          6636 non-null object  
id             6636 non-null int64  
manufacturer   5195 non-null object  
dtypes: int64(1), object(3)  
memory usage: 207.5+ KB
```

You can see the distribution of food groups with value_counts:

```
In [165]: pd.value_counts(info.group)[:10]
```

```
Out[165]:  
Vegetables and Vegetable Products    812  
Beef Products                        618  
Baked Products                       496  
Breakfast Cereals                   403  
Fast Foods                          365  
Legumes and Legume Products         365  
Lamb, Veal, and Game Products       345  
Sweets                              341  
Pork Products                       328  
Fruits and Fruit Juices              328  
Name: group, dtype: int64
```



미국농무부 영양소 정보

USDA Food Database

Now, to do some analysis on all of the nutrient data, it's easiest to assemble the nutrients for each food into a single large table. To do so, we need to take several steps. First, I'll convert each list of food nutrients to a DataFrame, add a column for the food id, and append the DataFrame to a list. Then, these can be concatenated together with concat:

If all goes well, nutrients should look like this:

```
In [167]: nutrients
Out[167]:
```

	description	group	units	value	id
0	Protein	Composition	g	25.180	1008
1	Total lipid (fat)	Composition	g	29.200	1008
2	Carbohydrate, by difference	Composition	g	3.060	1008
3	Ash	Other	g	3.280	1008
4	Energy	Energy	kcal	376.000	1008
...	...	...	...	...	...
389350	Vitamin B-12, added	Vitamins	mcg	0.000	43546
389351	Cholesterol	Other	mg	0.000	43546
389352	Fatty acids, total saturated	Other	g	0.072	43546
389353	Fatty acids, total monounsaturated	Other	g	0.028	43546
389354	Fatty acids, total polyunsaturated	Other	g	0.041	43546

[389355 rows x 5 columns]

I noticed that there are duplicates in this DataFrame, so it makes things easier to drop them:

```
In [168]: nutrients.duplicated().sum() # number of duplicates
Out[168]: 14179

In [169]: nutrients = nutrients.drop_duplicates()
```

Since 'group' and 'description' are in both DataFrame objects, we can rename for clarity:

```
In [170]: col_mapping = {'description' : 'food',
.....:                  'group'       : 'fgroup'}

In [171]: info = info.rename(columns=col_mapping, copy=False)

In [172]: info.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6636 entries, 0 to 6635
Data columns (total 4 columns):
food          6636 non-null object
fgroup        6636 non-null object
id            6636 non-null int64
manufacturer  5195 non-null object
dtypes: int64(1), object(3)
memory usage: 207.5+ KB
```

```
In [173]: col_mapping = {'description' : 'nutrient',
.....:                  'group'       : 'nutgroup'}
```

```
In [174]: nutrients = nutrients.rename(columns=col_mapping, copy=False)
```

```
In [175]: nutrients
Out[175]:
```

	nutrient	nutgroup	units	value	id
0	Protein	Composition	g	25.180	1008
1	Total lipid (fat)	Composition	g	29.200	1008
2	Carbohydrate, by difference	Composition	g	3.060	1008
3	Ash	Other	g	3.280	1008
4	Energy	Energy	kcal	376.000	1008
...	...	...	...	...	...
389350	Vitamin B-12, added	Vitamins	mcg	0.000	43546
389351	Cholesterol	Other	mg	0.000	43546
389352	Fatty acids, total saturated	Other	g	0.072	43546
389353	Fatty acids, total monounsaturated	Other	g	0.028	43546
389354	Fatty acids, total polyunsaturated	Other	g	0.041	43546

[375176 rows x 5 columns]



미국농무부 영양소 정보

USDA Food Database

With all of this done, we're ready to merge info with nutrients:

```
In [176]: ndata = pd.merge(nutrients, info, on='id', how='outer')
```

```
In [177]: ndata.info()
<class 'pandas.core.frame.DataFrame'>
Int64Index: 375176 entries, 0 to 375175
Data columns (total 8 columns):
nutrient      375176 non-null object
nutgroup      375176 non-null object
units         375176 non-null object
value         375176 non-null float64
id            375176 non-null int64
food          375176 non-null object
fgroup        375176 non-null object
manufacturer  293854 non-null object
dtypes: float64(1), int64(1), object(6)
memory usage: 25.8+ MB
```

```
In [178]: ndata.iloc[30000]
```

```
Out[178]:
          nutrient      Glycine
          nutgroup      Amino Acids
          units          g
          value          0.04
          id          6158
          food      Soup, tomato bisque, canned, condensed
          fgroup      Soups, Sauces, and Gravies
          manufacturer
          Name: 30000, dtype: object
```

We could now make a plot of median values by food group and nutrient type (see Figure 14-11):

```
In [180]: result = ndata.groupby(['nutrient', 'fgroup'])['value'].quantile(0.5)
```

```
In [181]: result['Zinc, Zn'].sort_values().plot(kind='barh')
```

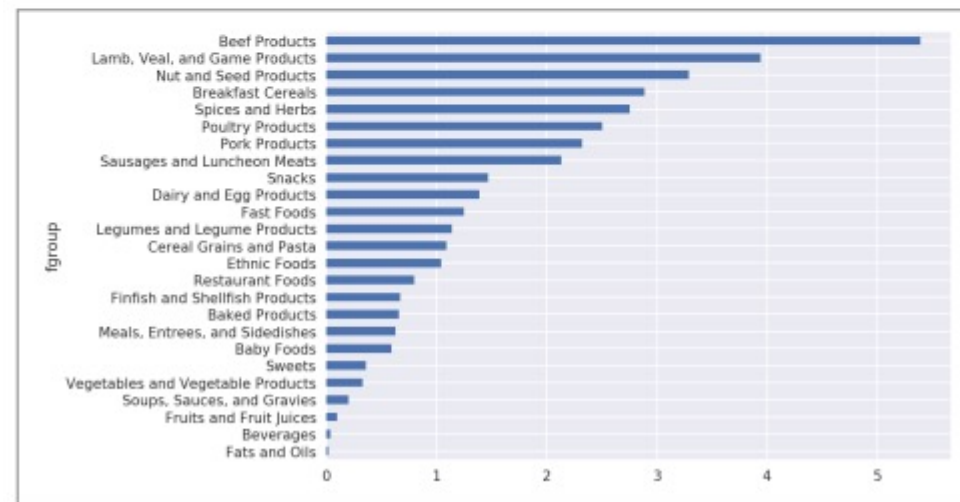


Figure 14-11. Median zinc values by nutrient group



미국농무부 영양소 정보

USDA Food Database

With a little cleverness, you can find which food is most dense in each nutrient:

```
by_nutrient = ndata.groupby(['nutgroup', 'nutrient'])

get_maximum = lambda x: x.loc[x.value.idxmax()]
get_minimum = lambda x: x.loc[x.value.idxmin()]

max_foods = by_nutrient.apply(get_maximum)[['value', 'food']]

# make the food a little smaller
max_foods.food = max_foods.food.str[:50]
```

The resulting DataFrame is a bit too large to display in the book; here is only the 'Amino Acids' nutrient group:

```
In [183]: max_foods.loc['Amino Acids']['food']
Out[183]:
nutrient
Alanine      Gelatins, dry powder, unsweetened
Arginine     Seeds, sesame flour, low-fat
Aspartic acid      Soy protein isolate
Cystine           Seeds, cottonseed flour, low fat (glandless)
Glutamic acid     Soy protein isolate
...
Serine           Soy protein isolate, PROTEIN TECHNOLOGIES INTE...
Threonine        Soy protein isolate, PROTEIN TECHNOLOGIES INTE...
Tryptophan       Sea lion, Steller, meat with fat (Alaska Native)
Tyrosine         Soy protein isolate, PROTEIN TECHNOLOGIES INTE...
Valine           Soy protein isolate, PROTEIN TECHNOLOGIES INTE...
Name: food, Length: 19, dtype: object
```



2012년 연방선거관리위원회 데이터베이스

2012 Federal Election Commission Database

14.5 2012 Federal Election Commission Database

The US Federal Election Commission publishes data on contributions to political campaigns. This includes contributor names, occupation and employer, address, and contribution amount. An interesting dataset is from the 2012 US presidential election. A version of the dataset I downloaded in June 2012 is a 150 megabyte CSV file `P00000001-ALL.csv` (see the book's data repository), which can be loaded with `pd.read_csv`:

```
In [184]: fec = pd.read_csv('datasets/fec/P00000001-ALL.csv')
```

```
In [185]: fec.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1001731 entries, 0 to 1001730
Data columns (total 16 columns):
cmte_id      1001731 non-null object
cand_id      1001731 non-null object
cand_nm      1001731 non-null object
contbr_nm    1001731 non-null object
contbr_city  1001712 non-null object
contbr_st    1001727 non-null object
contbr_zip   1001620 non-null object
contbr_employer  988002 non-null object
contbr_occupation  993301 non-null object
contb_receipt_amt  1001731 non-null float64
contb_receipt_dt  1001731 non-null object
receipt_desc  14166 non-null object
memo_cd      92482 non-null object
memo_text    97770 non-null object
form_tp      1001731 non-null object
file_num     1001731 non-null int64
dtypes: float64(1), int64(1), object(14)
memory usage: 122.3+ MB
```

A sample record in the DataFrame looks like this:

```
In [186]: fec.iloc[123456]
Out[186]:
cmte_id      C00431445
cand_id      P80003338
cand_nm      Obama, Barack
contbr_nm    ELLMAN, IRA
contbr_city   TEMPE

receipt_desc      NaN
memo_cd           NaN
memo_text         NaN
form_tp          SA17A
file_num         772372
Name: 123456, Length: 16, dtype: object
```

You may think of some ways to start slicing and dicing this data to extract informative statistics about donors and patterns in the campaign contributions. I'll show you a number of different analyses that apply techniques in this book.

You can see that there are no political party affiliations in the data, so this would be useful to add. You can get a list of all the unique political candidates using `unique`:

```
In [187]: unique_cands = fec.cand_nm.unique()

In [188]: unique_cands
Out[188]:
array(['Bachmann, Michelle', 'Romney, Mitt', 'Obama, Barack',
      'Roemer, Charles E. 'Buddy' III', 'Pawlenty, Timothy',
      'Johnson, Gary Earl', 'Paul, Ron', 'Santorum, Rick', 'Cain, Herman',
      'Gingrich, Newt', 'McCotter, Thaddeus G', 'Huntsman, Jon',
      'Perry, Rick'], dtype=object)

In [189]: unique_cands[2]
Out[189]: 'Obama, Barack'
```



2012년 연방선거관리위원회 데이터베이스

2012 Federal Election Commission Database

One way to indicate party affiliation is using a dict:¹

```
parties = {'Bachmann, Michelle': 'Republican',
           'Cain, Herman': 'Republican',
           'Gingrich, Newt': 'Republican',
           'Huntsman, Jon': 'Republican',
           'Johnson, Gary Earl': 'Republican',
           'McCotter, Thaddeus G': 'Republican',
           'Obama, Barack': 'Democrat',
           'Paul, Ron': 'Republican',
           'Pawlenty, Timothy': 'Republican',
           'Perry, Rick': 'Republican',
           'Roemer, Charles E. 'Buddy' III': 'Republican',
           'Romney, Mitt': 'Republican',
           'Santorum, Rick': 'Republican'}
```

Now, using this mapping and the `map` method on Series objects, you can compute an array of political parties from the candidate names:

```
In [191]: fec.cand_nm[123456:123461]
Out[191]:
123456    Obama, Barack
123457    Obama, Barack
123458    Obama, Barack
123459    Obama, Barack
123460    Obama, Barack
Name: cand_nm, dtype: object
```

```
In [192]: fec.cand_nm[123456:123461].map(parties)
Out[192]:
123456    Democrat
123457    Democrat
123458    Democrat
123459    Democrat
123460    Democrat
Name: cand_nm, dtype: object
```

```
# Add it as a column
In [193]: fec['party'] = fec.cand_nm.map(parties)

In [194]: fec['party'].value_counts()
Out[194]:
Democrat      593746
Republican    407985
Name: party, dtype: int64
```

A couple of data preparation points. First, this data includes both contributions and refunds (negative contribution amount):

```
In [195]: (fec.contb_receipt_amt > 0).value_counts()
Out[195]:
True      991475
False     10256
Name: contb_receipt_amt, dtype: int64
```

To simplify the analysis, I'll restrict the dataset to positive contributions:

```
In [196]: fec = fec[fec.contb_receipt_amt > 0]
```

Since Barack Obama and Mitt Romney were the main two candidates, I'll also prepare a subset that just has contributions to their campaigns:

```
In [197]: fec_mrbo = fec[fec.cand_nm.isin(['Obama, Barack', 'Romney, Mitt'])]
```



2012년 연방선거관리위원회 데이터베이스 > 직업 및 고용주에 따른 기부 통계

2012 Federal Election Commission Database

Donation Statistics by Occupation and Employer

Donations by occupation is another oft-studied statistic. For example, lawyers (attorneys) tend to donate more money to Democrats, while business executives tend to donate more to Republicans. You have no reason to believe me; you can see for yourself in the data. First, the total number of donations by occupation is easy:

```
In [198]: fec.contbr_occupation.value_counts()[:10]
Out[198]:
RETIRED                233990
INFORMATION REQUESTED  35107
ATTORNEY               34286
HOMEMAKER              29931
PHYSICIAN              23432
INFORMATION REQUESTED PER BEST EFFORTS  21138
ENGINEER               14334
TEACHER               13990
CONSULTANT             13273
PROFESSOR              12555
Name: contbr_occupation, dtype: int64
```

You will notice by looking at the occupations that many refer to the same basic job type, or there are several variants of the same thing. The following code snippet illustrates a technique for cleaning up a few of them by mapping from one occupation to another; note the “trick” of using `dict.get` to allow occupations with no mapping to “pass through”:

```
occ_mapping = {
    'INFORMATION REQUESTED PER BEST EFFORTS' : 'NOT PROVIDED',
    'INFORMATION REQUESTED' : 'NOT PROVIDED',
    'INFORMATION REQUESTED (BEST EFFORTS)' : 'NOT PROVIDED',
    'C.E.O.': 'CEO'
}

# If no mapping provided, return x
f = lambda x: occ_mapping.get(x, x)
fec.contbr_occupation = fec.contbr_occupation.map(f)
```

I'll also do the same thing for employers:

```
emp_mapping = {
    'INFORMATION REQUESTED PER BEST EFFORTS' : 'NOT PROVIDED',
    'INFORMATION REQUESTED' : 'NOT PROVIDED',
    'SELF' : 'SELF-EMPLOYED',
    'SELF EMPLOYED' : 'SELF-EMPLOYED',
}

# If no mapping provided, return x
f = lambda x: emp_mapping.get(x, x)
fec.contbr_employer = fec.contbr_employer.map(f)
```

Now, you can use `pivot_table` to aggregate the data by party and occupation, then filter down to the subset that donated at least \$2 million overall:

```
In [201]: by_occupation = fec.pivot_table('contb_receipt_amt',
.....:                                     index='contbr_occupation',
.....:                                     columns='party', aggfunc='sum')

In [202]: over_2mm = by_occupation[by_occupation.sum(1) > 2000000]

In [203]: over_2mm
Out[203]:
party          Democrat      Republican
contbr_occupation
ATTORNEY          11141982.97  7.477194e+06
CEO                2074974.79  4.211041e+06
CONSULTANT         2459912.71  2.544725e+06
ENGINEER            951525.55  1.818374e+06
EXECUTIVE          1355161.05  4.138850e+06
...
PRESIDENT          1878509.95  4.720924e+06
PROFESSOR           2165071.08  2.967027e+05
REAL ESTATE         528902.09  1.625902e+06
RETIRED             25305116.38  2.356124e+07
SELF-EMPLOYED       672393.40  1.640253e+06
[17 rows x 2 columns]
```



2012년 연방선거관리위원회 데이터베이스 > 직업 및 고용주에 따른 기부 통계

2012 Federal Election Commission Database

It can be easier to look at this data graphically as a bar plot ('barh' means horizontal bar plot; see Figure 14-12):

```
In [205]: over_2m.plot(kind='barh')
```

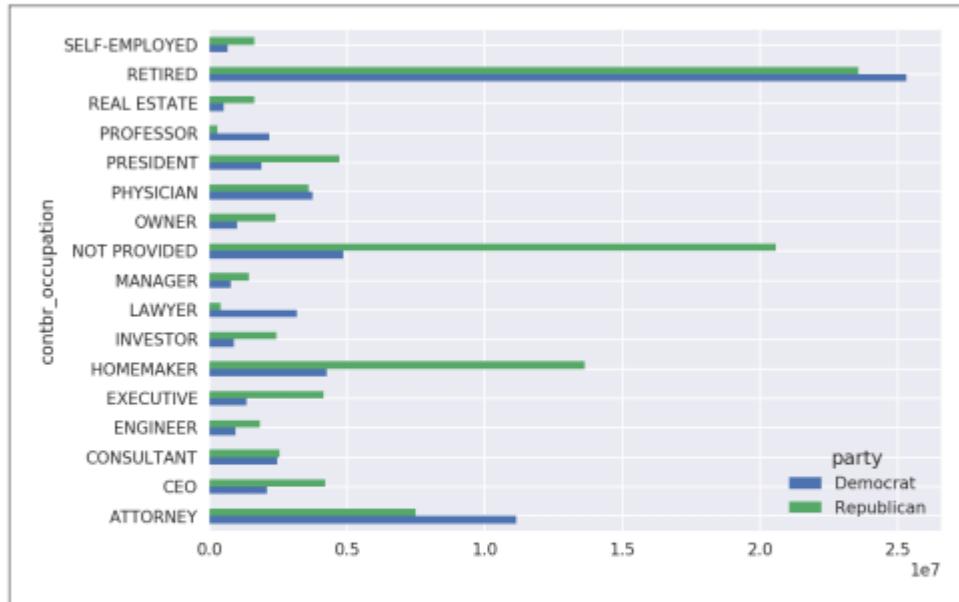


Figure 14-12. Total donations by party for top occupations

You might be interested in the top donor occupations or top companies that donated to Obama and Romney. To do this, you can group by candidate name and use a variant of the top method from earlier in the chapter:

```
def get_top_amounts(group, key, n=5):
    totals = group.groupby(key)['contb_receipt_amt'].sum()
    return totals.nlargest(n)
```

Then aggregate by occupation and employer:

```
In [207]: grouped = fec_mrbo.groupby('cand_nm')
```

```
In [208]: grouped.apply(get_top_amounts, 'contbr_occupation', n=7)
```

```
Out[208]:
```

cand_nm	contbr_occupation	contb_receipt_amt
Obama, Barack	RETIRED	25305116.38
	ATTORNEY	11141982.97
	INFORMATION REQUESTED	4866973.96
	HOMEMAKER	4248875.80
	PHYSICIAN	3735124.94

...

cand_nm	contbr_employer	contb_receipt_amt
Romney, Mitt	HOMEMAKER	8147446.22
	ATTORNEY	5364718.82
	PRESIDENT	2491244.89
	EXECUTIVE	2300947.03
	C.E.O.	1968386.11

Name: contb_receipt_amt, Length: 14, dtype: float64

```
In [209]: grouped.apply(get_top_amounts, 'contbr_employer', n=10)
```

```
Out[209]:
```

cand_nm	contbr_employer	contb_receipt_amt
Obama, Barack	RETIRED	22694358.85
	SELF-EMPLOYED	17080985.96
	NOT EMPLOYED	8586308.70
	INFORMATION REQUESTED	5053480.37
	HOMEMAKER	2605408.54

...

cand_nm	contbr_employer	contb_receipt_amt
Romney, Mitt	CREDIT SUISSE	281150.00
	MORGAN STANLEY	267266.00
	GOLDMAN SACH & CO.	238250.00
	BARCLAYS CAPITAL	162750.00
	H.I.G. CAPITAL	139500.00

Name: contb_receipt_amt, Length: 20, dtype: float64



2012년 연방선거관리위원회 데이터베이스 > 기부금액

2012 Federal Election Commission Database

Bucketing Donation Amounts

A useful way to analyze this data is to use the cut function to discretize the contributor amounts into buckets by contribution size:

```
In [210]: bins = np.array([0, 1, 10, 100, 1000, 10000,
.....:                    100000, 1000000, 10000000])

In [211]: labels = pd.cut(fec_mrbo.contb_receipt_amt, bins)

In [212]: labels
Out[212]:
411      (10, 100]
412     (100, 1000]
413     (100, 1000]
414      (10, 100]
415      (10, 100]
...
701381   (10, 100]
701382   (100, 1000]
701383      (1, 10]
701384   (10, 100]
701385   (100, 1000]
Name: contb_receipt_amt, Length: 694282, dtype: category
Categories (8, interval[int64]): [(0, 1] < (1, 10] < (10, 100] < (100, 1000] < (1000, 10000] <
                                (10000, 100000] < (100000, 1000000] < (1000000,
                                10000000]]
```

We can then group the data for Obama and Romney by name and bin label to get a histogram by donation size:

```
In [213]: grouped = fec_mrbo.groupby(['cand_nm', labels])
```

```
In [214]: grouped.size().unstack(0)
Out[214]:
```

cand_nm contb_receipt_amt	Obama, Barack	Romney, Mitt
(0, 1]	493.0	77.0
(1, 10]	40070.0	3681.0
(10, 100]	372280.0	31853.0
(100, 1000]	153991.0	43357.0
(1000, 10000]	22284.0	26186.0
(10000, 100000]	2.0	1.0
(100000, 1000000]	3.0	NaN
(1000000, 10000000]	4.0	NaN

This data shows that Obama received a significantly larger number of small donations than Romney. You can also sum the contribution amounts and normalize within buckets to visualize percentage of total donations of each size by candidate (Figure 14-13 shows the resulting plot):

```
In [216]: bucket_sums = grouped.contb_receipt_amt.sum().unstack(0)
```

```
In [217]: normed_sums = bucket_sums.div(bucket_sums.sum(axis=1), axis=0)
```

```
In [218]: normed_sums
Out[218]:
```

cand_nm contb_receipt_amt	Obama, Barack	Romney, Mitt
(0, 1]	0.805182	0.194818
(1, 10]	0.918767	0.081233
(10, 100]	0.910769	0.089231
(100, 1000]	0.710176	0.289824
(1000, 10000]	0.447326	0.552674
(10000, 100000]	0.823120	0.176880
(100000, 1000000]	1.000000	NaN
(1000000, 10000000]	1.000000	NaN



2012년 연방선거관리위원회 데이터베이스 > 기부금액

2012 Federal Election Commission Database

```
In [219]: normed_sums[:-2].plot(kind='barh')
```

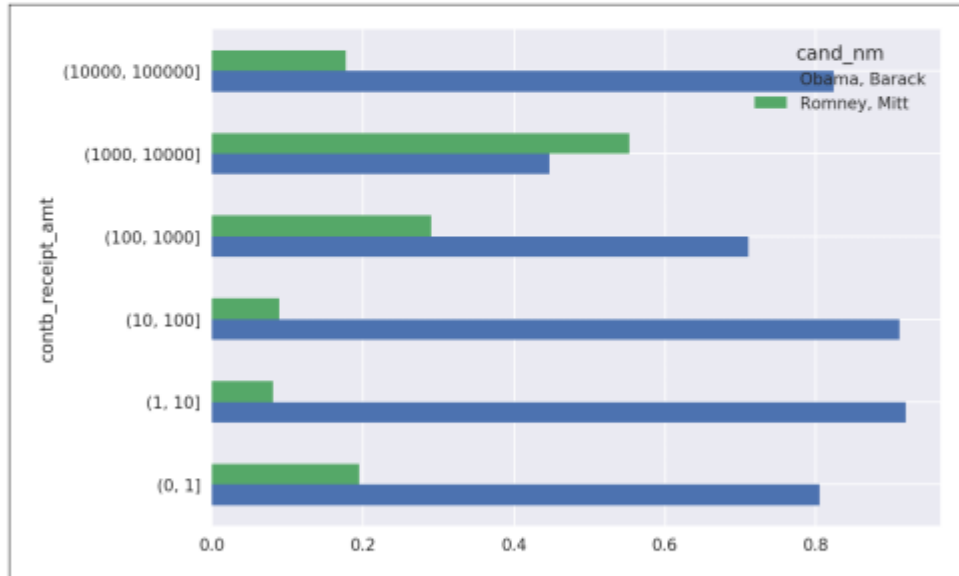


Figure 14-13. Percentage of total donations received by candidates for each donation size

I excluded the two largest bins as these are not donations by individuals.

This analysis can be refined and improved in many ways. For example, you could aggregate donations by donor name and zip code to adjust for donors who gave many small amounts versus one or more large donations. I encourage you to download and explore the dataset yourself.



2012년 연방선거관리위원회 데이터베이스 > 주별 기부 통계

2012 Federal Election Commission Database

Donation Statistics by State

Aggregating the data by candidate and state is a routine affair:

```
In [220]: grouped = fec_mrbo.groupby(['cand_nm', 'contbr_st'])

In [221]: totals = grouped.contb_receipt_amt.sum().unstack(0).fillna(0)

In [222]: totals = totals[totals.sum(1) > 100000]

In [223]: totals[:10]
Out[223]:
cand_nm  Obama, Barack  Romney, Mitt
contbr_st
AK          281840.15      86204.24
AL          543123.48     527303.51
AR          359247.28     105556.00
AZ          1506476.98    1888436.23
CA          23824984.24   11237636.60
CO          2132429.49    1506714.12
CT          2068291.26    3499475.45
DC          4373538.80    1025137.50
DE           336669.14      82712.00
FL          7318178.58    8338458.81
```

If you divide each row by the total contribution amount, you get the relative percentage of total donations by state for each candidate:

```
In [224]: percent = totals.div(totals.sum(1), axis=0)

In [225]: percent[:10]
Out[225]:
cand_nm  Obama, Barack  Romney, Mitt
contbr_st
AK          0.765778      0.234222
AL          0.507390      0.492610
AR          0.772902      0.227098
AZ          0.443745      0.556255
CA          0.679498      0.320502
CO          0.585970      0.414030
CT          0.371476      0.628524
DC          0.810113      0.189887
DE          0.802776      0.197224
FL          0.467417      0.532583
```



고급 NumPy

Advanced NumPy

- ndarray 객체 구조
- 고급 배열 조작 기법
- 브로드캐스팅
- 고급 ufunc
- 구조화된 배열과 레코드 배열
- 정렬 알고리즘



ndarray 객체 구조

ndarray Object Internals

A.1 ndarray Object Internals

The NumPy ndarray provides a means to interpret a block of homogeneous data (either contiguous or strided) as a multidimensional array object. The data type, or *dtype*, determines how the data is interpreted as being floating point, integer, boolean, or any of the other types we've been looking at.

Part of what makes ndarray flexible is that every array object is a *strided* view on a block of data. You might wonder, for example, how the array view `arr[:, :2, ::-1]` does not copy any data. The reason is that the ndarray is more than just a chunk of memory and a dtype; it also has “striding” information that enables the array to move through memory with varying step sizes. More precisely, the ndarray internally consists of the following:

- A *pointer to data*—that is, a block of data in RAM or in a memory-mapped file
- The *data type* or dtype, describing fixed-size value cells in the array
- A tuple indicating the array's *shape*
- A tuple of *strides*, integers indicating the number of bytes to “step” in order to advance one element along a dimension

See [Figure A-1](#) for a simple mockup of the ndarray innards.

For example, a 10×5 array would have shape `(10, 5)`:

```
In [10]: np.ones((10, 5)).shape  
Out[10]: (10, 5)
```

A typical (C order) $3 \times 4 \times 5$ array of float64 (8-byte) values has strides `(160, 40, 8)` (knowing about the strides can be useful because, in general, the larger the strides on a particular axis, the more costly it is to perform computation along that axis):

```
In [11]: np.ones((3, 4, 5), dtype=np.float64).strides  
Out[11]: (160, 40, 8)
```

While it is rare that a typical NumPy user would be interested in the array strides, they are the critical ingredient in constructing “zero-copy” array views. Strides can even be negative, which enables an array to move “backward” through memory (this would be the case, for example, in a slice like `obj[::-1]` or `obj[:, ::-1]`).

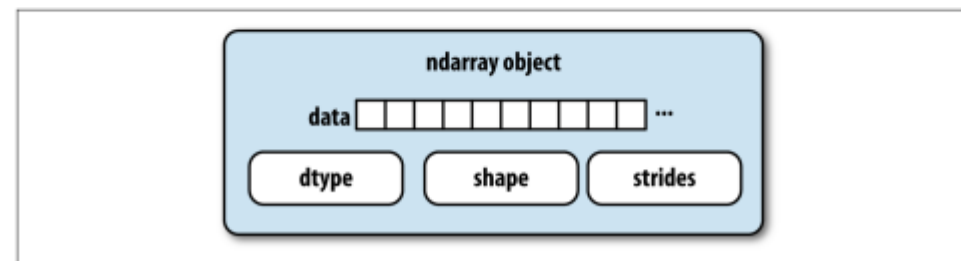


Figure A-1. The NumPy ndarray object



ndarray 객체 구조 > NumPy dtype 구조

ndarray Object Internals

NumPy dtype Hierarchy

You may occasionally have code that needs to check whether an array contains integers, floating-point numbers, strings, or Python objects. Because there are multiple types of floating-point numbers (float16 through float128), checking that the dtype is among a list of types would be very verbose. Fortunately, the dtypes have super-classes such as `np.integer` and `np.floating`, which can be used in conjunction with the `np.issubdtype` function:

```
In [12]: ints = np.ones(10, dtype=np.uint16)

In [13]: floats = np.ones(10, dtype=np.float32)

In [14]: np.issubdtype(ints.dtype, np.integer)
Out[14]: True

In [15]: np.issubdtype(floats.dtype, np.floating)
Out[15]: True
```

You can see all of the parent classes of a specific dtype by calling the type's `mro` method:

```
In [16]: np.float64.mro()
Out[16]:
[numpy.float64,
 numpy.floating,
 numpy.inexact,
 numpy.number,
 numpy.generic,
 float,
 object]
```

Therefore, we also have:

```
In [17]: np.issubdtype(ints.dtype, np.number)
Out[17]: True
```

Most NumPy users will never have to know about this, but it occasionally comes in handy. See [Figure A-2](#) for a graph of the dtype hierarchy and parent-subclass relationships.¹

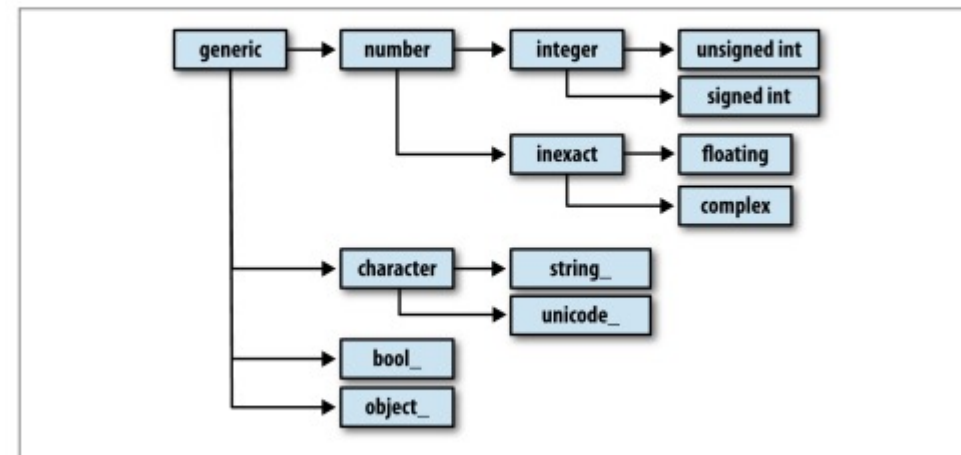


Figure A-2. The NumPy dtype class hierarchy



고급 배열 조작 기법 > 배열 재형성하기

Advanced Array Manipulation

A.2 Advanced Array Manipulation

There are many ways to work with arrays beyond fancy indexing, slicing, and boolean subsetting. While much of the heavy lifting for data analysis applications is handled by higher-level functions in pandas, you may at some point need to write a data algorithm that is not found in one of the existing libraries.

Reshaping Arrays

In many cases, you can convert an array from one shape to another without copying any data. To do this, pass a tuple indicating the new shape to the `reshape` array instance method. For example, suppose we had a one-dimensional array of values that we wished to rearrange into a matrix (the result is shown in **Figure A-3**):

```
In [18]: arr = np.arange(8)

In [19]: arr
Out[19]: array([0, 1, 2, 3, 4, 5, 6, 7])

In [20]: arr.reshape((4, 2))
Out[20]:
array([[0, 1],
       [2, 3],
       [4, 5],
       [6, 7]])
```

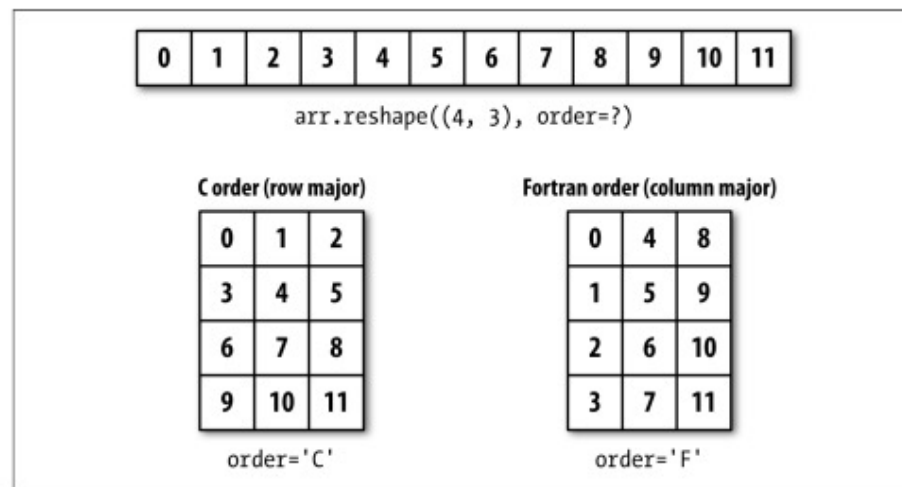


Figure A-3. Reshaping in C (row major) or Fortran (column major) order

A multidimensional array can also be reshaped:

```
In [21]: arr.reshape((4, 2)).reshape((2, 4))
Out[21]:
array([[0, 1, 2, 3],
       [4, 5, 6, 7]])
```

One of the passed shape dimensions can be `-1`, in which case the value used for that dimension will be inferred from the data:

```
In [22]: arr = np.arange(15)

In [23]: arr.reshape((5, -1))
Out[23]:
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11],
       [12, 13, 14]])
```



고급 배열 조작 기법 > 배열 재형성하기

Advanced Array Manipulation

Since an array's `shape` attribute is a tuple, it can be passed to `reshape`, too:

```
In [24]: other_arr = np.ones((3, 5))

In [25]: other_arr.shape
Out[25]: (3, 5)

In [26]: arr.reshape(other_arr.shape)
Out[26]:
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14]])
```

The opposite operation of `reshape` from one-dimensional to a higher dimension is typically known as *flattening* or *raveling*:

```
In [27]: arr = np.arange(15).reshape((5, 3))

In [28]: arr
Out[28]:
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11],
       [12, 13, 14]])

In [29]: arr.ravel()
Out[29]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14])
```

`ravel` does not produce a copy of the underlying values if the values in the result were contiguous in the original array. The `flatten` method behaves like `ravel` except it always returns a copy of the data:

```
In [30]: arr.flatten()
Out[30]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14])
```

The data can be reshaped or raveled in different orders. This is a slightly nuanced topic for new NumPy users and is therefore the next subtopic.



고급 배열 조작 기법 > 배열 이어붙이고 나누기

Advanced Array Manipulation

Concatenating and Splitting Arrays

`numpy.concatenate` takes a sequence (tuple, list, etc.) of arrays and joins them together in order along the input axis:

```
In [35]: arr1 = np.array([[1, 2, 3], [4, 5, 6]])

In [36]: arr2 = np.array([[7, 8, 9], [10, 11, 12]])

In [37]: np.concatenate([arr1, arr2], axis=0)
Out[37]:
array([[ 1,  2,  3],
       [ 4,  5,  6],
       [ 7,  8,  9],
       [10, 11, 12]])

In [38]: np.concatenate([arr1, arr2], axis=1)
Out[38]:
array([[ 1,  2,  3,  7,  8,  9],
       [ 4,  5,  6, 10, 11, 12]])
```

There are some convenience functions, like `vstack` and `hstack`, for common kinds of concatenation. The preceding operations could have been expressed as:

```
In [39]: np.vstack((arr1, arr2))
Out[39]:
array([[ 1,  2,  3],
       [ 4,  5,  6],
       [ 7,  8,  9],
       [10, 11, 12]])

In [40]: np.hstack((arr1, arr2))
Out[40]:
array([[ 1,  2,  3,  7,  8,  9],
       [ 4,  5,  6, 10, 11, 12]])
```

`split`, on the other hand, slices apart an array into multiple arrays along an axis:

```
In [41]: arr = np.random.randn(5, 2)

In [42]: arr
Out[42]:
array([[ -0.2047,  0.4789],
       [ -0.5194, -0.5557],
       [  1.9658,  1.3934],
       [  0.0929,  0.2817],
       [  0.769 ,  1.2464]])

In [43]: first, second, third = np.split(arr, [1, 3])

In [44]: first
Out[44]: array([[ -0.2047,  0.4789]])

In [45]: second
Out[45]:
array([[ -0.5194, -0.5557],
       [  1.9658,  1.3934]])

In [46]: third
Out[46]:
array([[ 0.0929,  0.2817],
       [ 0.769 ,  1.2464]])
```

The value `[1, 3]` passed to `np.split` indicate the indices at which to split the array into pieces.

See [Table A-1](#) for a list of all relevant concatenation and splitting functions, some of which are provided only as a convenience of the very general-purpose `concatenate`.

Table A-1. Array concatenation functions

Function	Description
<code>concatenate</code>	Most general function, concatenates collection of arrays along one axis
<code>vstack</code> , <code>row_stack</code>	Stack arrays row-wise (along axis 0)
<code>hstack</code>	Stack arrays column-wise (along axis 1)
<code>column_stack</code>	Like <code>hstack</code> , but converts 1D arrays to 2D column vectors first
<code>dstack</code>	Stack arrays "depth"-wise (along axis 2)
<code>split</code>	Split array at passed locations along a particular axis
<code>hsplit</code> / <code>vsplit</code>	Convenience functions for splitting on axis 0 and 1, respectively

Repeating Elements: *tile* and *repeat*

Two useful tools for repeating or replicating arrays to produce larger arrays are the *repeat* and *tile* functions. *repeat* replicates each element in an array some number of times, producing a larger array:

```
In [53]: arr = np.arange(3)

In [54]: arr
Out[54]: array([0, 1, 2])

In [55]: arr.repeat(3)
Out[55]: array([0, 0, 0, 1, 1, 1, 2, 2, 2])
```



The need to replicate or repeat arrays can be less common with NumPy than it is with other array programming frameworks like MATLAB. One reason for this is that *broadcasting* often fills this need better, which is the subject of the next section.

By default, if you pass an integer, each element will be repeated that number of times. If you pass an array of integers, each element can be repeated a different number of times:

```
In [56]: arr.repeat([2, 3, 4])
Out[56]: array([0, 0, 1, 1, 1, 2, 2, 2, 2])
```

Multidimensional arrays can have their elements repeated along a particular axis.

```
In [57]: arr = np.random.randn(2, 2)

In [58]: arr
Out[58]:
array([[ -2.0016,  -0.3718],
       [  1.669 ,  -0.4386]])

In [59]: arr.repeat(2, axis=0)
Out[59]:
array([[ -2.0016,  -0.3718],
       [ -2.0016,  -0.3718],
       [  1.669 ,  -0.4386],
       [  1.669 ,  -0.4386]])
```

Note that if no axis is passed, the array will be flattened first, which is likely not what you want. Similarly, you can pass an array of integers when repeating a multidimensional array to repeat a given slice a different number of times:

```
In [60]: arr.repeat([2, 3], axis=0)
Out[60]:
array([[ -2.0016,  -0.3718],
       [ -2.0016,  -0.3718],
       [  1.669 ,  -0.4386],
       [  1.669 ,  -0.4386],
       [  1.669 ,  -0.4386]])

In [61]: arr.repeat([2, 3], axis=1)
Out[61]:
array([[ -2.0016,  -2.0016,  -0.3718,  -0.3718,  -0.3718],
       [  1.669 ,   1.669 ,  -0.4386,  -0.4386,  -0.4386]])
```

tile, on the other hand, is a shortcut for stacking copies of an array along an axis. Visually you can think of it as being akin to “laying down tiles”:

```
In [62]: arr
Out[62]:
array([[ -2.0016,  -0.3718],
       [  1.669 ,  -0.4386]])

In [63]: np.tile(arr, 2)
Out[63]:
array([[ -2.0016,  -0.3718,  -2.0016,  -0.3718],
       [  1.669 ,  -0.4386,   1.669 ,  -0.4386]])
```



고급 배열 조작 기법 > 원소 반복하기: *repeat*와 *tile*

Advanced Array Manipulation

The second argument is the number of tiles; with a scalar, the tiling is made row by row, rather than column by column. The second argument to `tile` can be a tuple indicating the layout of the “tiling”:

```
In [64]: arr
Out[64]:
array([[ -2.0016,  -0.3718],
       [  1.669 ,  -0.4386]])

In [65]: np.tile(arr, (2, 1))
Out[65]:
array([[ -2.0016,  -0.3718],
       [  1.669 ,  -0.4386],
       [ -2.0016,  -0.3718],
       [  1.669 ,  -0.4386]])

In [66]: np.tile(arr, (3, 2))
Out[66]:
array([[ -2.0016,  -0.3718,  -2.0016,  -0.3718],
       [  1.669 ,  -0.4386,   1.669 ,  -0.4386],
       [ -2.0016,  -0.3718,  -2.0016,  -0.3718],
       [  1.669 ,  -0.4386,   1.669 ,  -0.4386],
       [ -2.0016,  -0.3718,  -2.0016,  -0.3718],
       [  1.669 ,  -0.4386,   1.669 ,  -0.4386]])
```



고급 배열 조작 기법 > 팬시 색인: *take*와 *put*

Advanced Array Manipulation

Fancy Indexing Equivalents: take and put

As you may recall from **Chapter 4**, one way to get and set subsets of arrays is by *fancy* indexing using integer arrays:

```
In [67]: arr = np.arange(10) * 100  
  
In [68]: inds = [7, 1, 2, 6]  
  
In [69]: arr[inds]  
Out[69]: array([700, 100, 200, 600])
```

There are alternative ndarray methods that are useful in the special case of only making a selection on a single axis:

```
In [70]: arr.take(inds)  
Out[70]: array([700, 100, 200, 600])  
  
In [71]: arr.put(inds, 42)  
  
In [72]: arr  
Out[72]: array([  0,  42,  42, 300, 400, 500,  42,  42, 800, 900])  
  
In [73]: arr.put(inds, [40, 41, 42, 43])  
  
In [74]: arr  
Out[74]: array([  0,  41,  42, 300, 400, 500,  43,  40, 800, 900])
```

To use *take* along other axes, you can pass the *axis* keyword:

```
In [75]: inds = [2, 0, 2, 1]  
  
In [76]: arr = np.random.randn(2, 4)  
  
In [77]: arr  
Out[77]:  
array([[ -0.5397,  0.477 ,  3.2489, -1.0212],  
       [-0.5771,  0.1241,  0.3026,  0.5238]])  
  
In [78]: arr.take(inds, axis=1)  
Out[78]:  
array([[ 3.2489, -0.5397,  3.2489,  0.477 ],  
       [ 0.3026, -0.5771,  0.3026,  0.1241]])
```



A.3 Broadcasting

Broadcasting describes how arithmetic works between arrays of different shapes. It can be a powerful feature, but one that can cause confusion, even for experienced users. The simplest example of broadcasting occurs when combining a scalar value with an array:

```
In [79]: arr = np.arange(5)

In [80]: arr
Out[80]: array([0, 1, 2, 3, 4])

In [81]: arr * 4
Out[81]: array([ 0,  4,  8, 12, 16])
```

Here we say that the scalar value 4 has been *broadcast* to all of the other elements in the multiplication operation.

For example, we can demean each column of an array by subtracting the column means. In this case, it is very simple:

```
In [82]: arr = np.random.randn(4, 3)

In [83]: arr.mean(0)
Out[83]: array([-0.3928, -0.3824, -0.8768])

In [84]: demeaned = arr - arr.mean(0)

In [85]: demeaned
Out[85]:
array([[ 0.3937,  1.7263,  0.1633],
       [-0.4384, -1.9878, -0.9839],
       [-0.468 ,  0.9426, -0.3891],
       [ 0.5126, -0.6811,  1.2097]])

In [86]: demeaned.mean(0)
Out[86]: array([-0.,  0., -0.])
```

See [Figure A-4](#) for an illustration of this operation. Demeaning the rows as a broadcast operation requires a bit more care. Fortunately, broadcasting potentially lower dimensional values across any dimension of an array (like subtracting the row means from each column of a two-dimensional array) is possible as long as you follow the rules.

This brings us to:

The Broadcasting Rule

Two arrays are compatible for broadcasting if for each *trailing dimension* (i.e., starting from the end) the axis lengths match or if either of the lengths is 1. Broadcasting is then performed over the missing or length 1 dimensions.

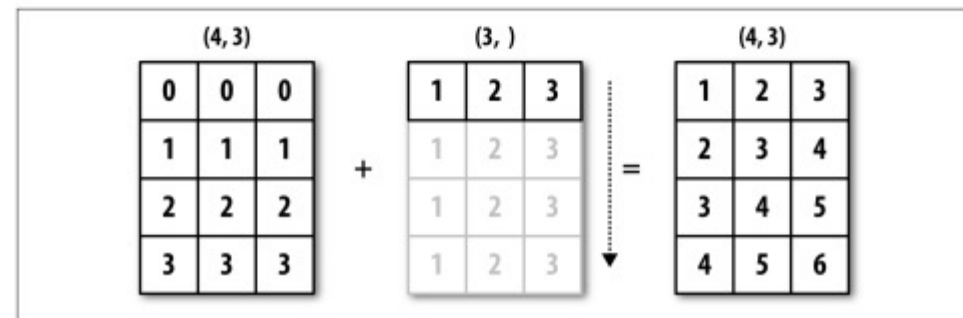


Figure A-4. Broadcasting over axis 0 with a 1D array



브로드캐스팅

Broadcasting

Even as an experienced NumPy user, I often find myself having to pause and draw a diagram as I think about the broadcasting rule. Consider the last example and suppose we wished instead to subtract the mean value from each row. Since `arr.mean(0)` has length 3, it is compatible for broadcasting across axis 0 because the trailing dimension in `arr` is 3 and therefore matches. According to the rules, to subtract over axis 1 (i.e., subtract the row mean from each row), the smaller array must have shape (4, 1):

```
In [87]: arr
Out[87]:
array([[ 0.0009,  1.3438, -0.7135],
       [-0.8312, -2.3702, -1.8608],
       [-0.8608,  0.5601, -1.2659],
       [ 0.1198, -1.0635,  0.3329]])
```

```
In [88]: row_means = arr.mean(1)
```

```
In [89]: row_means.shape
Out[89]: (4,)
```

```
In [90]: row_means.reshape((4, 1))
Out[90]:
array([[ 0.2104],
       [-1.6874],
       [-0.5222],
       [-0.2036]])
```

```
In [91]: demeaned = arr - row_means.reshape((4, 1))
```

```
In [92]: demeaned.mean(1)
Out[92]: array([ 0., -0.,  0.,  0.])
```

See [Figure A-5](#) for an illustration of this operation.

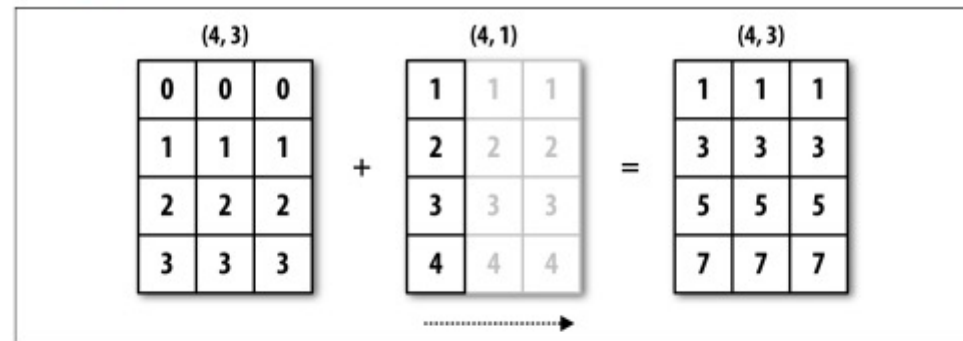


Figure A-5. Broadcasting over axis 1 of a 2D array

See [Figure A-6](#) for another illustration, this time adding a two-dimensional array to a three-dimensional one across axis 0.

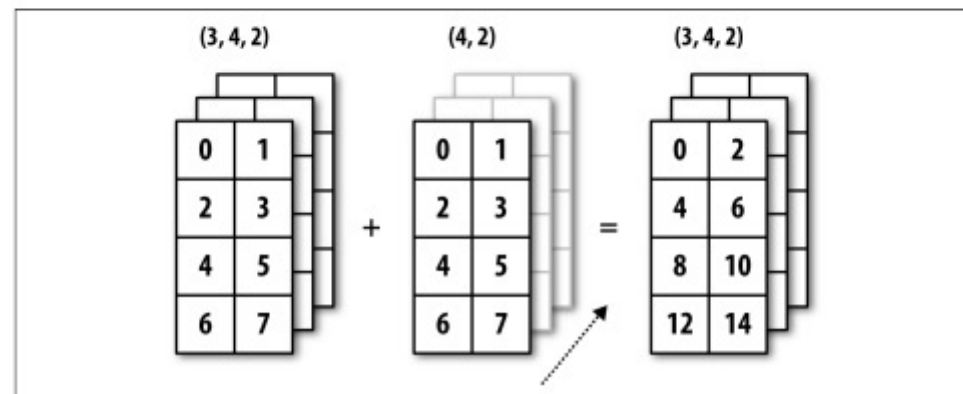


Figure A-6. Broadcasting over axis 0 of a 3D array



브로드캐스팅 > 다른 축에 대해 브로드캐스팅하기

Broadcasting

Broadcasting Over Other Axes

Broadcasting with higher dimensional arrays can seem even more mind-bending, but it is really a matter of following the rules. If you don't, you'll get an error like this:

```
In [93]: arr = arr.mean(1)
-----
ValueError                                Traceback (most recent call last)
<ipython-input-93-7b87b85a20b2> in <module>()
----> 1 arr = arr.mean(1)
ValueError: operands could not be broadcast together with shapes (4,3) (4,)
```

It's quite common to want to perform an arithmetic operation with a lower dimensional array across axes other than axis 0. According to the broadcasting rule, the "broadcast dimensions" must be 1 in the smaller array. In the example of row demeaning shown here, this meant reshaping the row means to be shape (4, 1) instead of (4,):

```
In [94]: arr = arr.mean(1).reshape((4, 1))
Out[94]:
array([[ -0.2095,  1.1334, -0.9239],
       [  0.8562, -0.6828, -0.1734],
       [-0.3386,  1.0823, -0.7438],
       [  0.3234, -0.8599,  0.5365]])
```

In the three-dimensional case, broadcasting over any of the three dimensions is only a matter of reshaping the data to be shape-compatible. Figure A-7 nicely visualizes the shapes required to broadcast over each axis of a three-dimensional array.

A common problem, therefore, is needing to add a new axis with length 1 specifically for broadcasting purposes. Using reshape is one option, but inserting an axis requires constructing a tuple indicating the new shape. This can often be a tedious exercise. Thus, NumPy arrays offer a special syntax for inserting new axes by indexing. We use the special np.newaxis attribute along with "full" slices to insert the new axis:

```
In [95]: arr = np.zeros((4, 4))

In [96]: arr_3d = arr[:, np.newaxis, :]

In [97]: arr_3d.shape
Out[97]: (4, 1, 4)

In [98]: arr_1d = np.random.normal(size=3)

In [99]: arr_1d[:, np.newaxis]
Out[99]:
array([[ -2.3594],
       [-0.1995],
       [-1.542 ]])

In [100]: arr_1d[np.newaxis, :]
Out[100]: array([[ -2.3594, -0.1995, -1.542 ]])
```



브로드캐스팅 > 다른 축에 대해 브로드캐스팅하기

Broadcasting

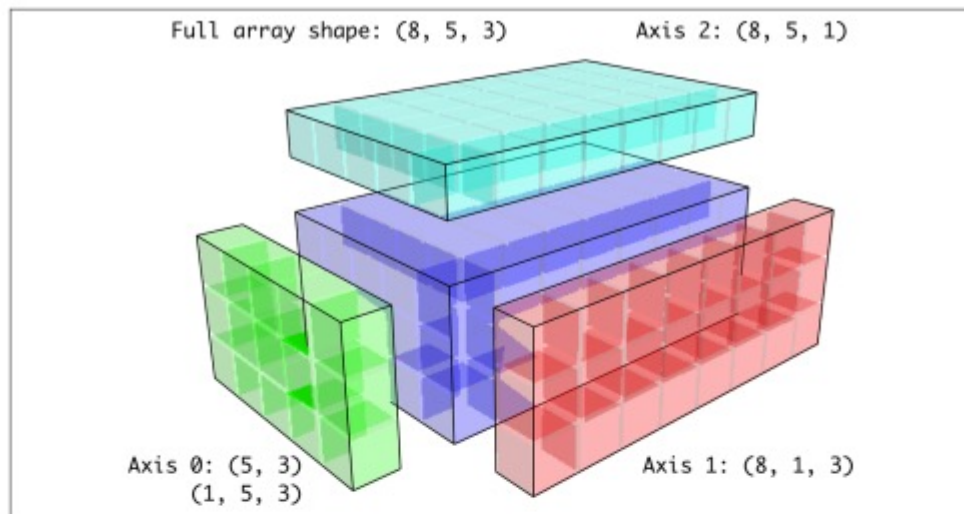


Figure A-7. Compatible 2D array shapes for broadcasting over a 3D array

Thus, if we had a three-dimensional array and wanted to demean axis 2, say, we would need to write:

```
In [101]: arr = np.random.randn(3, 4, 5)

In [102]: depth_means = arr.mean(2)

In [103]: depth_means
Out[103]:
array([[ -0.4735,  0.3971, -0.0228,  0.2001],
       [-0.3521, -0.281 , -0.071 , -0.1586],
       [ 0.6245,  0.6047,  0.4396, -0.2846]])

In [104]: depth_means.shape
Out[104]: (3, 4)

In [105]: demeaned = arr - depth_means[:, :, np.newaxis]

In [106]: demeaned.mean(2)
Out[106]:
array([[ 0.,  0., -0., -0.],
       [ 0.,  0., -0.,  0.],
       [ 0.,  0., -0., -0.]])
```

You might be wondering if there's a way to generalize demeaning over an axis without sacrificing performance. There is, but it requires some indexing gymnastics:

```
def demean_axis(arr, axis=0):
    means = arr.mean(axis)

    # This generalizes things like[:, :, np.newaxis] to N dimensions
    indexer = [slice(None)] * arr.ndim
    indexer[axis] = np.newaxis
    return arr - means[indexer]
```



브로드캐스팅 > 브로드캐스팅을 이용해서 배열에 값 대입하기

Broadcasting

Setting Array Values by Broadcasting

The same broadcasting rule governing arithmetic operations also applies to setting values via array indexing. In a simple case, we can do things like:

```
In [107]: arr = np.zeros((4, 3))
```

```
In [108]: arr[:] = 5
```

```
In [109]: arr
```

```
Out[109]:
```

```
array([[ 5.,  5.,  5.],
       [ 5.,  5.,  5.],
       [ 5.,  5.,  5.],
       [ 5.,  5.,  5.]])
```

However, if we had a one-dimensional array of values we wanted to set into the columns of the array, we can do that as long as the shape is compatible:

```
In [110]: col = np.array([1.28, -0.42, 0.44, 1.6])
```

```
In [111]: arr[:, np.newaxis] = col[:, np.newaxis]
```

```
In [112]: arr
```

```
Out[112]:
```

```
array([[ 1.28,  1.28,  1.28],
       [-0.42, -0.42, -0.42],
       [ 0.44,  0.44,  0.44],
       [ 1.6 ,  1.6 ,  1.6 ]])
```

```
In [113]: arr[:2] = [[-1.37], [0.509]]
```

```
In [114]: arr
```

```
Out[114]:
```

```
array([[ -1.37, -1.37, -1.37 ],
       [ 0.509,  0.509,  0.509],
       [ 0.44 ,  0.44 ,  0.44 ],
       [ 1.6 ,  1.6 ,  1.6 ]])
```



고급 ufunc 사용법 > ufunc 인스턴스 메서드

Advanced ufunc Usage

A.4 Advanced ufunc Usage

While many NumPy users will only make use of the fast element-wise operations provided by the universal functions, there are a number of additional features that occasionally can help you write more concise code without loops.

ufunc Instance Methods

Each of NumPy's binary ufuncs has special methods for performing certain kinds of special vectorized operations. These are summarized in [Table A-2](#), but I'll give a few concrete examples to illustrate how they work.

`reduce` takes a single array and aggregates its values, optionally along an axis, by performing a sequence of binary operations. For example, an alternative way to sum elements in an array is to use `np.add.reduce`:

```
In [115]: arr = np.arange(10)
```

```
In [116]: np.add.reduce(arr)
Out[116]: 45
```

```
In [117]: arr.sum()
Out[117]: 45
```

The starting value (0 for `add`) depends on the ufunc. If an axis is passed, the reduction is performed along that axis. This allows you to answer certain kinds of questions in a concise way. As a less trivial example, we can use `np.logical_and` to check whether the values in each row of an array are sorted:

```
In [118]: np.random.seed(12346) # for reproducibility
```

```
In [119]: arr = np.random.randn(5, 5)
```

```
In [120]: arr[:,2].sort(1) # sort a few rows
```

```
In [121]: arr[:, :-1] < arr[:, 1:]
Out[121]:
array([[ True,  True,  True,  True],
       [False,  True, False, False],
       [ True,  True,  True,  True],
       [ True, False,  True,  True],
       [ True,  True,  True,  True]], dtype=bool)
```

```
In [122]: np.logical_and.reduce(arr[:, :-1] < arr[:, 1:], axis=1)
Out[122]: array([ True, False,  True, False,  True], dtype=bool)
```

Note that `logical_and.reduce` is equivalent to the `all` method.

`accumulate` is related to `reduce` like `cumsum` is related to `sum`. It produces an array of the same size with the intermediate "accumulated" values:

```
In [123]: arr = np.arange(15).reshape((3, 5))
```

```
In [124]: np.add.accumulate(arr, axis=1)
Out[124]:
array([[ 0,  1,  3,  6, 10],
       [ 5, 11, 18, 26, 35],
       [10, 21, 33, 46, 60]])
```

`outer` performs a pairwise cross-product between two arrays:

```
In [125]: arr = np.arange(3).repeat([1, 2, 2])
```

```
In [126]: arr
Out[126]: array([0, 1, 1, 2, 2])
```

```
In [127]: np.multiply.outer(arr, np.arange(5))
Out[127]:
array([[0, 0, 0, 0, 0],
       [0, 1, 2, 3, 4],
       [0, 1, 2, 3, 4],
       [0, 2, 4, 6, 8],
       [0, 2, 4, 6, 8]])
```



고급 ufunc 사용법 > ufunc 인스턴스 메서드

Advanced ufunc Usage

The output of `outer` will have a dimension that is the sum of the dimensions of the inputs:

```
In [128]: x, y = np.random.randn(3, 4), np.random.randn(5)
```

```
In [129]: result = np.subtract.outer(x, y)
```

```
In [130]: result.shape
```

```
Out[130]: (3, 4, 5)
```

The last method, `reduceat`, performs a “local reduce,” in essence an array groupby operation in which slices of the array are aggregated together. It accepts a sequence of “bin edges” that indicate how to split and aggregate the values:

```
In [131]: arr = np.arange(10)
```

```
In [132]: np.add.reduceat(arr, [0, 5, 8])
```

```
Out[132]: array([10, 18, 17])
```

The results are the reductions (here, sums) performed over `arr[0:5]`, `arr[5:8]`, and `arr[8:]`. As with the other methods, you can pass an axis argument:

```
In [133]: arr = np.multiply.outer(np.arange(4), np.arange(5))
```

```
In [134]: arr
```

```
Out[134]:
```

```
array([[ 0,  0,  0,  0,  0],
       [ 0,  1,  2,  3,  4],
       [ 0,  2,  4,  6,  8],
       [ 0,  3,  6,  9, 12]])
```

```
In [135]: np.add.reduceat(arr, [0, 2, 4], axis=1)
```

```
Out[135]:
```

```
array([[ 0,  0,  0],
       [ 1,  5,  4],
       [ 2, 10,  8],
       [ 3, 15, 12]])
```

See [Table A-2](#) for a partial listing of ufunc methods.

Table A-2. ufunc methods

Method	Description
<code>reduce(x)</code>	Aggregate values by successive applications of the operation
<code>accumulate(x)</code>	Aggregate values, preserving all partial aggregates
<code>reduceat(x, bins)</code>	“Local” reduce or “group by”; reduce contiguous slices of data to produce aggregated array
<code>outer(x, y)</code>	Apply operation to all pairs of elements in <code>x</code> and <code>y</code> ; the resulting array has shape <code>x.shape + y.shape</code>



구조화된 배열과 레코드 배열

Structured and Record Arrays

A.5 Structured and Record Arrays

You may have noticed up until now that ndarray is a *homogeneous* data container; that is, it represents a block of memory in which each element takes up the same number of bytes, determined by the dtype. On the surface, this would appear to not allow you to represent heterogeneous or tabular-like data. A *structured* array is an ndarray in which each element can be thought of as representing a *struct* in C (hence the “structured” name) or a row in a SQL table with multiple named fields:

```
In [144]: dtype = [('x', np.float64), ('y', np.int32)]

In [145]: sarr = np.array([(1.5, 6), (np.pi, -2)], dtype=dtype)

In [146]: sarr
Out[146]:
array([( 1.5      , 6), ( 3.1416, -2)],
      dtype=[('x', '<f8'), ('y', '<i4')])
```

There are several ways to specify a structured dtype (see the online NumPy documentation). One typical way is as a list of tuples with (field_name, field_data_type). Now, the elements of the array are tuple-like objects whose elements can be accessed like a dictionary:

```
In [147]: sarr[0]
Out[147]: ( 1.5, 6)

In [148]: sarr[0]['y']
Out[148]: 6
```

The field names are stored in the dtype.names attribute. When you access a field on the structured array, a strided view on the data is returned, thus copying nothing:

```
In [149]: sarr['x']
Out[149]: array([ 1.5      ,  3.1416])
```



구조화된 배열과 레코드 배열 > 중첩된 dtype과 다차원 필드

Structured and Record Arrays

Nested dtypes and Multidimensional Fields

When specifying a structured dtype, you can additionally pass a shape (as an int or tuple):

```
In [150]: dtype = [('x', np.int64, 3), ('y', np.int32)]

In [151]: arr = np.zeros(4, dtype=dtype)

In [152]: arr
Out[152]:
array([(0, 0, 0), 0], ([0, 0, 0], 0), ([0, 0, 0], 0), ([0, 0, 0], 0)],
      dtype=[('x', '<i8', (3,)), ('y', '<i4')])
```

In this case, the x field now refers to an array of length 3 for each record:

```
In [153]: arr[0]['x']
Out[153]: array([0, 0, 0])
```

Conveniently, accessing `arr['x']` then returns a two-dimensional array instead of a one-dimensional array as in prior examples:

```
In [154]: arr['x']
Out[154]:
array([[0, 0, 0],
       [0, 0, 0],
       [0, 0, 0],
       [0, 0, 0]])
```

This enables you to express more complicated, nested structures as a single block of memory in an array. You can also nest dtypes to make more complex structures. Here is an example:

```
In [155]: dtype = [('x', [(('a', 'f8'), ('b', 'f4'))], ('y', np.int32)]

In [156]: data = np.array([((1, 2), 5), ((3, 4), 6)], dtype=dtype)

In [157]: data['x']
Out[157]:
array([(1., 2.), (3., 4.)],
      dtype=[('a', '<f8'), ('b', '<f4')])

In [158]: data['y']
Out[158]: array([5, 6], dtype=int32)

In [159]: data['x']['a']
Out[159]: array([1., 3.])
```

pandas DataFrame does not support this feature directly, though it is similar to hierarchical indexing.



구조화된 배열과 레코드 배열 > 구조화된 배열을 써야 하는 이유

Structured and Record Arrays

Why Use Structured Arrays?

Compared with, say, a pandas DataFrame, NumPy structured arrays are a comparatively low-level tool. They provide a means to interpreting a block of memory as a tabular structure with arbitrarily complex nested columns. Since each element in the array is represented in memory as a fixed number of bytes, structured arrays provide a very fast and efficient way of writing data to and from disk (including memory maps), transporting it over the network, and other such uses.

As another common use for structured arrays, writing data files as fixed-length record byte streams is a common way to serialize data in C and C++ code, which is commonly found in legacy systems in industry. As long as the format of the file is known (the size of each record and the order, byte size, and data type of each element), the data can be read into memory with `np.fromfile`. Specialized uses like this are beyond the scope of this book, but it's worth knowing that such things are possible.



정렬에 관하여

More About Sorting

A.6 More About Sorting

Like Python's built-in list, the ndarray sort instance method is an *in-place* sort, meaning that the array contents are rearranged without producing a new array:

```
In [160]: arr = np.random.randn(6)
```

```
In [161]: arr.sort()
```

```
In [162]: arr
```

```
Out[162]: array([-1.082 ,  0.3759,  0.8014,  1.1397,  1.2888,  1.8413])
```

When sorting arrays in-place, remember that if the array is a view on a different ndarray, the original array will be modified:

```
In [163]: arr = np.random.randn(3, 5)
```

```
In [164]: arr
```

```
Out[164]:
```

```
array([[ -0.3318, -1.4711,  0.8705, -0.0847, -1.1329],
       [-1.0111, -0.3436,  2.1714,  0.1234, -0.0189],
       [ 0.1773,  0.7424,  0.8548,  1.038 , -0.329 ]])
```

```
In [165]: arr[:, 0].sort() # Sort first column values in-place
```

```
In [166]: arr
```

```
Out[166]:
```

```
array([[ -1.0111, -1.4711,  0.8705, -0.0847, -1.1329],
       [-0.3318, -0.3436,  2.1714,  0.1234, -0.0189],
       [ 0.1773,  0.7424,  0.8548,  1.038 , -0.329 ]])
```

On the other hand, `numpy.sort` creates a new, sorted copy of an array. Otherwise, it accepts the same arguments (such as `kind`) as `ndarray.sort`:

```
In [167]: arr = np.random.randn(5)
```

```
In [168]: arr
```

```
Out[168]: array([-1.1181, -0.2415, -2.0051,  0.7379, -1.0614])
```

```
In [169]: np.sort(arr)
```

```
Out[169]: array([-2.0051, -1.1181, -1.0614, -0.2415,  0.7379])
```

```
In [170]: arr
```

```
Out[170]: array([-1.1181, -0.2415, -2.0051,  0.7379, -1.0614])
```

All of these sort methods take an `axis` argument for sorting the sections of data along the passed axis independently:

```
In [171]: arr = np.random.randn(3, 5)
```

```
In [172]: arr
```

```
Out[172]:
```

```
array([[ 0.5955, -0.2682,  1.3389, -0.1872,  0.9111],
       [-0.3215,  1.0054, -0.5168,  1.1925, -0.1989],
       [ 0.3969, -1.7638,  0.6071, -0.2222, -0.2171]])
```

```
In [173]: arr.sort(axis=1)
```

```
In [174]: arr
```

```
Out[174]:
```

```
array([[ -0.2682, -0.1872,  0.5955,  0.9111,  1.3389],
       [-0.5168, -0.3215, -0.1989,  1.0054,  1.1925],
       [-1.7638, -0.2222, -0.2171,  0.3969,  0.6071]])
```

You may notice that none of the sort methods have an option to sort in descending order. This is a problem in practice because array slicing produces views, thus not producing a copy or requiring any computational work. Many Python users are familiar with the “trick” that for a list `values`, `values[::-1]` returns a list in reverse order. The same is true for ndarrays:

```
In [175]: arr[:, ::-1]
```

```
Out[175]:
```

```
array([[ 1.3389,  0.9111,  0.5955, -0.1872, -0.2682],
       [ 1.1925,  1.0054, -0.1989, -0.3215, -0.5168],
       [ 0.6071,  0.3969, -0.2171, -0.2222, -1.7638]])
```



정렬에 관하여 > 간접 정렬: argsort와 lexsort

More About Sorting

Indirect Sorts: argsort and lexsort

In data analysis you may need to reorder datasets by one or more keys. For example, a table of data about some students might need to be sorted by last name, then by first name. This is an example of an *indirect* sort, and if you've read the pandas-related chapters you have already seen many higher-level examples. Given a key or keys (an array of values or multiple arrays of values), you wish to obtain an array of integer *indices* (I refer to them colloquially as *indexers*) that tells you how to reorder the data to be in sorted order. Two methods for this are argsort and numpy.lexsort. As an example:

```
In [176]: values = np.array([5, 0, 1, 3, 2])

In [177]: indexer = values.argsort()

In [178]: indexer
Out[178]: array([1, 2, 4, 3, 0])

In [179]: values[indexer]
Out[179]: array([0, 1, 2, 3, 5])
```

As a more complicated example, this code reorders a two-dimensional array by its first row:

```
In [180]: arr = np.random.randn(3, 5)

In [181]: arr[0] = values

In [182]: arr
Out[182]:
array([[ 5.      ,  0.      ,  1.      ,  3.      ,  2.      ],
       [-0.3636, -0.1378,  2.1777, -0.4728,  0.8356],
       [-0.2089,  0.2316,  0.728 , -1.3918,  1.9956]])

In [183]: arr[:, arr[0].argsort()]
Out[183]:
array([[ 0.      ,  1.      ,  2.      ,  3.      ,  5.      ],
       [-0.1378,  2.1777,  0.8356, -0.4728, -0.3636],
       [ 0.2316,  0.728 ,  1.9956, -1.3918, -0.2089]])
```

lexsort is similar to argsort, but it performs an indirect *lexicographical* sort on multiple key arrays. Suppose we wanted to sort some data identified by first and last names:

```
In [184]: first_name = np.array(['Bob', 'Jane', 'Steve', 'Bill', 'Barbara'])

In [185]: last_name = np.array(['Jones', 'Arnold', 'Arnold', 'Jones', 'Walters'])

In [186]: sorter = np.lexsort((first_name, last_name))

In [187]: sorter
Out[187]: array([1, 2, 3, 0, 4])

In [188]: zip(last_name[sorter], first_name[sorter])
Out[188]: <zip at 0x7fa203eda1c8>
```

lexsort can be a bit confusing the first time you use it because the order in which the keys are used to order the data starts with the *last* array passed. Here, last_name was used before first_name.



pandas methods like Series's and DataFrame's sort_values method are implemented with variants of these functions (which also must take into account missing values).



정렬에 관하여 > 대안 정렬 알고리즘

More About Sorting

Alternative Sort Algorithms

A *stable* sorting algorithm preserves the relative position of equal elements. This can be especially important in indirect sorts where the relative ordering is meaningful:

```
In [189]: values = np.array(['2:first', '2:second', '1:first', '1:second',
.....:                     '1:third'])

In [190]: key = np.array([2, 2, 1, 1, 1])

In [191]: indexer = key.argsort(kind='mergesort')

In [192]: indexer
Out[192]: array([2, 3, 4, 0, 1])

In [193]: values.take(indexer)
Out[193]:
array(['1:first', '1:second', '1:third', '2:first', '2:second'],
      dtype='<U8')
```

The only stable sort available is *mergesort*, which has guaranteed $O(n \log n)$ performance (for complexity buffs), but its performance is on average worse than the default quicksort method. See [Table A-3](#) for a summary of available methods and their relative performance (and performance guarantees). This is not something that most users will ever have to think about, but it's useful to know that it's there.

Table A-3. Array sorting methods

Kind	Speed	Stable	Work space	Worst case
'quicksort'	1	No	0	$O(n^2)$
'mergesort'	2	Yes	$n / 2$	$O(n \log n)$
'heapsort'	3	No	0	$O(n \log n)$



정렬에 관하여 > 배열 일부만 정렬하기

More About Sorting

Partially Sorting Arrays

One of the goals of sorting can be to determine the largest or smallest elements in an array. NumPy has optimized methods, `numpy.partition` and `np.argpartition`, for partitioning an array around the k-th smallest element:

```
In [194]: np.random.seed(12345)
```

```
In [195]: arr = np.random.randn(20)
```

```
In [196]: arr
```

```
Out[196]:
```

```
array([-0.2047,  0.4789, -0.5194, -0.5557,  1.9658,  1.3934,  0.0929,  
        0.2817,  0.769 ,  1.2464,  1.0072, -1.2962,  0.275 ,  0.2289,  
        1.3529,  0.8864, -2.0016, -0.3718,  1.669 , -0.4386])
```

```
In [197]: np.partition(arr, 3)
```

```
Out[197]:
```

```
array([-2.0016, -1.2962, -0.5557, -0.5194, -0.3718, -0.4386, -0.2047,  
        0.2817,  0.769 ,  0.4789,  1.0072,  0.0929,  0.275 ,  0.2289,  
        1.3529,  0.8864,  1.3934,  1.9658,  1.669 ,  1.2464])
```

After you call `partition(arr, 3)`, the first three elements in the result are the smallest three values in no particular order. `numpy.argpartition`, similar to `numpy.argsort`, returns the indices that rearrange the data into the equivalent order:

```
In [198]: indices = np.argpartition(arr, 3)
```

```
In [199]: indices
```

```
Out[199]:
```

```
array([16, 11,  3,  2, 17, 19,  0,  7,  8,  1, 10,  6, 12, 13, 14, 15,  5,  
        4, 18,  9])
```

```
In [200]: arr.take(indices)
```

```
Out[200]:
```

```
array([-2.0016, -1.2962, -0.5557, -0.5194, -0.3718, -0.4386, -0.2047,  
        0.2817,  0.769 ,  0.4789,  1.0072,  0.0929,  0.275 ,  0.2289,  
        1.3529,  0.8864,  1.3934,  1.9658,  1.669 ,  1.2464])
```



정렬에 관하여 > *numpy.searchsorted*: 정렬된 배열에서 원소 찾기

More About Sorting

numpy.searchsorted: Finding Elements in a Sorted Array

searchsorted is an array method that performs a binary search on a sorted array, returning the location in the array where the value would need to be inserted to maintain sortedness:

```
In [201]: arr = np.array([0, 1, 7, 12, 15])
```

```
In [202]: arr.searchsorted(9)
Out[202]: 3
```

You can also pass an array of values to get an array of indices back:

```
In [203]: arr.searchsorted([0, 8, 11, 16])
Out[203]: array([0, 3, 3, 5])
```

You might have noticed that *searchsorted* returned 0 for the 0 element. This is because the default behavior is to return the index at the left side of a group of equal values:

```
In [204]: arr = np.array([0, 0, 0, 1, 1, 1, 1])
```

```
In [205]: arr.searchsorted([0, 1])
Out[205]: array([0, 3])
```

```
In [206]: arr.searchsorted([0, 1], side='right')
Out[206]: array([3, 7])
```

As another application of *searchsorted*, suppose we had an array of values between 0 and 10,000, and a separate array of “bucket edges” that we wanted to use to bin the data:

```
In [207]: data = np.floor(np.random.uniform(0, 10000, size=50))
```

```
In [208]: bins = np.array([0, 100, 1000, 5000, 10000])
```

```
In [209]: data
Out[209]:
array([[ 9940.,  6768.,  7908.,  1709.,   268.,  8003.,  9037.,   246.,
         4917.,  5262.,  5963.,   519.,  8950.,  7282.,  8183.,  5002.,
         8101.,   959.,  2189.,  2587.,  4681.,  4593.,  7095.,  1780.,
         5314.,  1677.,  7688.,  9281.,  6094.,  1501.,  4896.,  3773.,
         8486.,  9110.,  3838.,  3154.,  5683.,  1878.,  1258.,  6875.,
         7996.,  5735.,  9732.,  6340.,  8884.,  4954.,  3516.,  7142.,
         5039.,  2256.]])
```

To then get a labeling of which interval each data point belongs to (where 1 would mean the bucket [0, 100)), we can simply use *searchsorted*:

```
In [210]: labels = bins.searchsorted(data)
```

```
In [211]: labels
Out[211]:
array([4, 4, 4, 3, 2, 4, 4, 2, 3, 4, 4, 2, 4, 4, 4, 4, 4, 2, 3, 3, 3, 3, 4,
        3, 4, 3, 4, 4, 4, 3, 3, 3, 4, 4, 3, 3, 4, 3, 3, 4, 4, 4, 4, 4, 3,
        3, 4, 4, 3])
```

This, combined with pandas’s *groupby*, can be used to bin data:

```
In [212]: pd.Series(data).groupby(labels).mean()
Out[212]:
2    498.000000
3   3064.277778
4   7389.035714
dtype: float64
```

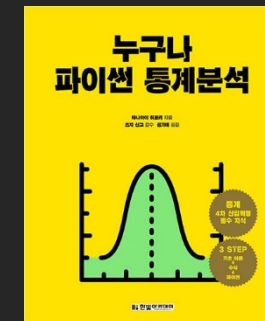
Q&A and Break Time

질의응답 및 휴식시간 (10분)



2부: 가설검정

- 1) 통계적 가설검정
- 2) 단측 검정 / 양측 검정
- 3) 가설검정 2가지 오류
- 4) 검정 – 정규분포의 모평균 (모분산 모를 때)
- 5) 검정 – 정규분포의 모분산
- 6) 2표본 문제 – 대응비교 t 검정
- 7) 2표본 문제 – 독립비교 t 검정
- 8) 2표본 문제 – 카이제곱 검정





가설 검정

1-1 통계적 가설검정



가설 검정

1-1. 통계적 가설검정

.

파이썬 기초 라이브러리 세팅

- Numpy (분포 구현)
- Pandas (분포 구현)
- Scipy의 stats 모듈 (통계)

```
1  import numpy as np
2  import pandas as pd
3  from scipy import stats
4
5  %precision 3
6  np.random.seed(1111)
```



가설 검증

1-1. 통계적 가설검정

.

파이썬 Scikit-learn 기반 기초 데이터 세팅 (RM: 가구당 평균 방 수)

7.1.1. Boston house prices dataset

Data Set Characteristics:

Number of Instances:	506
Number of Attributes:	13 numeric/categorical predictive. Median Value (attribute 14) is usually the target.
Attribute Information (in order):	• CRIM per capita crime rate by town • ZN proportion of residential land zoned for lots over 25,000 sq.ft. • INDUS proportion of non-retail business acres per town • CHAS Charles River dummy variable (= 1 if tract bounds river; 0 otherwise) • NOX nitric oxides concentration (parts per 10 million) • RM average number of rooms per dwelling • AGE proportion of owner-occupied units built prior to 1940 • DIS weighted distances to five Boston employment centres • RAD index of accessibility to radial highways • TAX full-value property-tax rate per \$10,000 • PTRATIO pupil-teacher ratio by town • B 1000(Bk - 0.63)^2 where Bk is the proportion of black people by town • LSTAT % lower status of the population • MEDV Median value of owner-occupied homes in \$1000's

```
1 from sklearn.datasets import load_boston
2
3 boston_houseprice_data = load_boston()
4
5 df = pd.DataFrame(
6     data = boston_houseprice_data.data,
7     columns = boston_houseprice_data.feature_names
8 )
9 rooms = np.array(df[ 'RM' ])
10 sample = np.array(df[ 'RM' ][:20])
```

```
1 p_mean, s_mean = np.mean(rooms), np.mean(sample)
2 p_mean, s_mean
(6.285, 6.192)
```

```
1 p_var, s_var = np.var(rooms), np.var(sample, ddof=1)
2 p_var, s_var
(0.493, 0.232)
```



가설 검정

1-1. 통계적 가설검정

.

통계적 가설검정(Statistical Hypothesis Testing)

- 모집단의 모수에 관하여 두 가지 가설을 세우고, 표본으로부터 계산되는 통계량을 이용하여 어느 가설이 옳은지 판단하는 통계적인 방법
- 가설검정 혹은 검정으로 불리기도 함



통계적 가설검정 예시

모집단이 정규분포를 따르고, 모분산이 0.2임을 알고 있다고 가정할 때,
보스톤의 '가구당 평균 방 수'가 6보다 큰지 여부를 알고 싶다고 하자

1. 모평균이 6이라는 가정을 먼저 한다
2. 그 가정을 바탕으로 20개의 표본은 $X_1, X_2, \dots, X_{20} \sim N(6, 0.2)$ 을 따른다
3. 표본평균 $\bar{X} \sim N(6, \frac{0.2}{20})$ 을 따른다
4. 표본평균 $\bar{X}$ 가 $P(\bar{X} \geq x) = 0.05$ 를 만족하는 x 를 찾는다
 $x = 6.164$ 가 되어 $P(\bar{X} \geq 6.164) = 0.05$ 이 된다

```
1 rv = stats.norm(6, np.sqrt(0.2/20))  
2 rv.isf(0.05)
```

```
6.164
```



가설 검정

1-1. 통계적 가설검정

.

통계적 가설검정 예시

- $P(\bar{X} \geq 6.164) = 0.05$ 이므로, 보스턴의 집 20개의 표본평균이 6.164를 넘는 경우는 5%의 확률로 발생한다. 즉, 표본평균이 6.192인 샘플의 경우는 5%의 확률로 발생하는 매우 드문 사건이라고 할 수 있다
- 즉, 샘플을 통해서 확인해보건데, 모평균이 6보다 더 큰 것이 우연이 아니라 실제로 더 크게 아닐까? 라는 생각으로 가설검정의 결론을 내리게 되는 것이 가설검정의 큰 흐름이다



통계적 가설검정 용어

- 2가지 가설

- 귀무가설(H_0 , Null Hypothesis) : 대립가설의 반대되는 개념.
- 대립가설(H_1 , Alternative Hypothesis) : 주장하고 싶은 가설.

주로 유의미한 차이가 있는지, 효과가 있는지를 주장하고자 함

(예시) 정책을 예시로 들면 **기존의 정책이 귀무가설**이며, 기존의 정책 대비 비용을 많이 들더라도 **새로운 정책**을 만들고 유의미한 정책임을 주장하고 싶을 때 대립가설로 설정한다.

- 귀무가설 기각/채택

- 귀무가설을 기각하다(Reject the null hypothesis): 귀무가설은 옳지 않다 (=> 새로운 정책이 유의미하다)
- 귀무가설을 채택하다(Accept the null hypothesis): 귀무가설이 옳지 않다고 할 수 없다

* 귀무가설이 옳다는 의미



통계적 가설검정 용어

- 유의하다 (Significant)

'귀무가설을 기각한다/채택한다'는 판단은 귀무가설을 바탕으로 표본을 통해 계산된 통계량이 특정 신뢰수준 범위 내에서 보기 드문 값인지 여부를 통해서 결정

- 1) 보기 드문 값이라면, 우연이 아닌 어떤 의미가 있는 유의미한 값으로 판단하여 귀무가설을 기각하고,
- 2) 그렇지 않다면, 귀무가설을 채택

즉, 우연이 아니라 어떤 의미가 있는 것을 유의하다라고 표현

(예시) 보스턴 주택의 평균 방 수가 '6보다 크다'라는 대립가설과 '6보다 작거나 같다'라는 귀무가설을 통해 모평균은 6으로 가정되며, 표본 추출시 95%의 확률로 표본평균 값이 $\bar{X} \leq 6.164$ 라는 조건을 만족하게 된다.

즉, 예시 데이터처럼 6.164 범위보다 바깥에 존재하는 표본평균인 **6.192**는 **보기 드문 값**이므로 **귀무가설은 기각**된다.



가설 검정

1-1. 통계적 가설검정

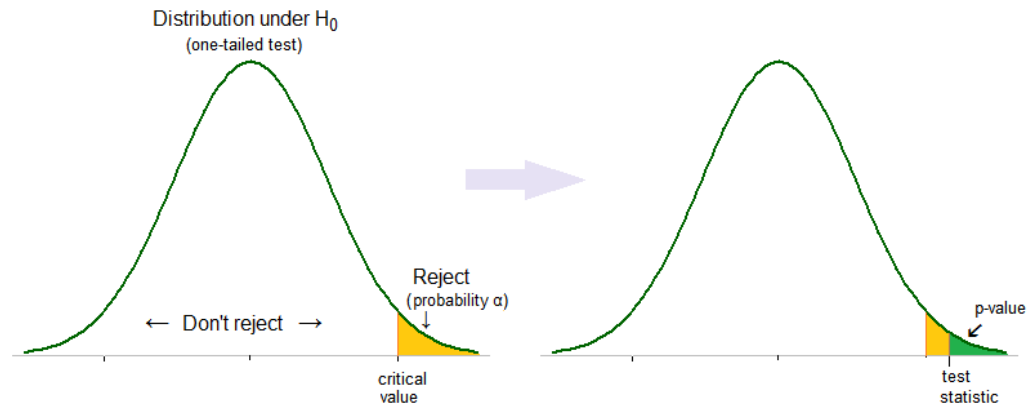
.

통계적 가설검정 용어

- 기각역/채택역 (예시 활용)
 - 기각역(Rejection Region) : 귀무가설이 기각되는 구간
($\bar{X} > 6.164$ 라면 귀무가설은 기각되므로 $\bar{X} > 6.164$ 가 기각역)
 - 채택역(Acceptance Region) : 귀무가설이 채택되는 구간
($\bar{X} \leq 6.164$ 라면 귀무가설이 채택되므로 $\bar{X} \leq 6.164$ 가 채택역)

통계적 가설검정 용어

- 유의수준(신뢰수준, Level of Significance)
: 표본평균이 기각역에 들어갈 확률 α
(주황색으로 칠해진 영역의 면적, 예시 = 0.05)
- 임계값 (Critical Value)
: 경계선상의 값 (예시 = 6.164)
- 검정통계량 (Test Statistic)
: 검정에 사용될 통계량 (예시 = 6.192)
- p값 (p-value)
: 검정통계량 기준으로 바깥 영역의 면적
(초록색으로 칠해진 영역의 면적, 예시 = 0.028)



```
1  rv.cdf(z)
0.972
```

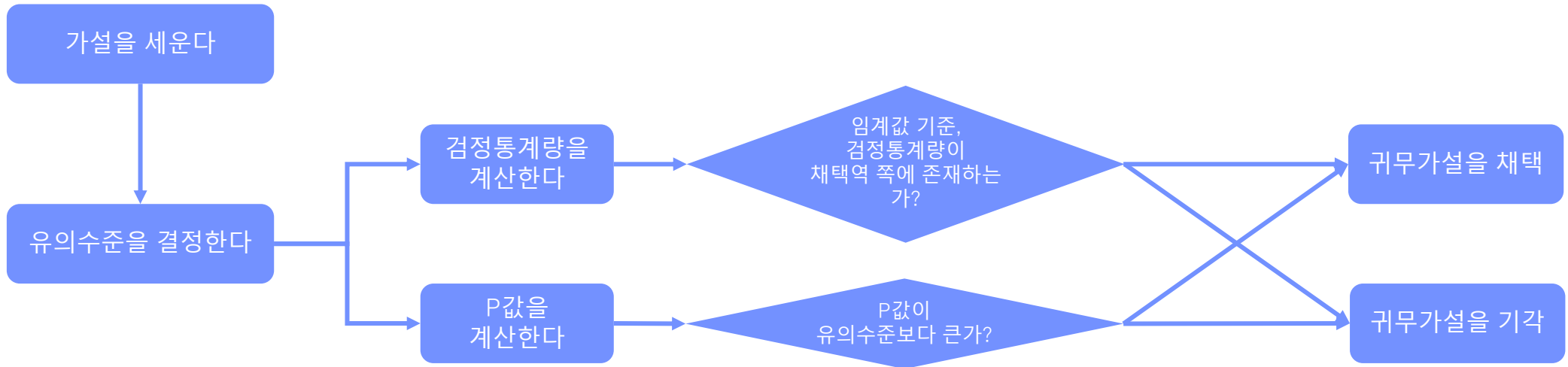


☒☒ ☒☒

1-1. 통계적 가설검정

.

☒☒☒☒ ☒☒☒





가설 검정

1-2 단측검정 & 양측검정



가설 검정

1-2. 단측검정 & 양측검정

.

단측검정 & 양측검정

- 단측검정 : 한 방향으로만 검정을 수행하는 방법
(예시) 대립가설이 모평균 >6 혹은 모평균 <6 일 때
- 양측검정 : 양방향으로 검정을 수행하는 방법
(예시) 대립가설이 모평균 $\neq 6$ 일 때

단측검정 & 양측검정 주의할 점

- 기각역이 다르다
 - 단측검정은 한쪽 방향으로 기각역 범위가 α 이다
 - 양측검정은 양쪽 방향으로 기각역 범위가 $\alpha/2$ 이다



가설 검정

1-3 가설검정, 두 가지 오류



가설검정의 두 가지 오류

- 제1종 오류 : 귀무가설이 옳을 때, 귀무가설을 기각하는 오류
(현 정책을 유지하는 것과 큰 차이 없음에도, 막대한 비용을 들여서 새로운 정책을 시도하는 오류)
 - 하지 말아야 할 것을 행한 것으로서 **오탐(False Positive)**이라 불리운다
 - 제1종 오류를 범할 확률은 **위험률**, α 라고 하며 유의수준과 일치한다
 - 분석가가 제어할 수 있는 오류 (주어진 유의수준에 맞추어 분석가가 의사결정을 정할 수 있음)
 - 제2종 오류 : 대립가설이 옳을 때, 귀무가설을 채택하는 오류
(새로운 정책으로의 변화가 유의미함에도, 큰 비용을 들이지 않는 현 정책을 고수하는 오류)
 - 행해야 할 것을 제대로 인식하지 못했으므로 **미탐(False Negative)**이라 불리운다
 - 제2종 오류를 범할 확률은 β 라고 하며, $1 - \beta$ 를 검정력(power)이라 불리운다
 - 분석가가 제어할 수 없는 오류 (분석가가 제어할 수 없는 비대칭 정보에 의하여, 귀무가설을 채택하게 된다)
- * **Positive:** 귀무가설 기각에 해당, **Negative:** 귀무가설 기각의 실패에 해당



가설 검정

1-3. 가설검정, 두 가지 오류

.

[Python] 가설검정의 두 가지 오류 계산

- 제1종 오류 : 평균이 6인데,
6보다 크다는 결론을 내리는 상황
- 제2종 오류 : 평균이 6이상인데,
6이상이라는 결론을 얻을 수 없는 상황

(비대칭)

```
1 rv = stats.norm(6, np.sqrt(0.2))

1 c = stats.norm().isf(0.05)
2 n_samples = 10000
3 cnt = 0
4 for _ in range(n_samples):
5     sample_ = np.round(rv.rvs(20), 2)
6     s_mean_ = np.mean(sample_)
7     z = (s_mean_ - 6) / np.sqrt(0.2/20)
8     if z > c:
9         cnt += 1
10 cnt / n_samples

0.048
```

```
1 rv = stats.norm(6.15, np.sqrt(0.2))

1 c = stats.norm().isf(0.05)
2 n_samples = 10000
3 cnt = 0
4 for _ in range(n_samples):
5     sample_ = np.round(rv.rvs(20), 2)
6     s_mean_ = np.mean(sample_)
7     z = (s_mean_ - 6) / np.sqrt(0.2/20)
8     if z <= c:
9         cnt += 1
10
11 cnt / n_samples

0.561
```

가설 검정

2-1 검정 – 정규분포의 모평균 (모분산 알 때)



☒☒ ☒☒

2-1. 검정 – 정규분포의 모평균 (모분산 알 때)

.

모분산을 알 때, 정규분포 모평균에 대한 검정

- $X_1, X_2, \dots, X_n$ 이 서로 독립이고 $N(\mu, \sigma^2)$ 인 확률분포를 따르며
모평균 μ 에 관한 유의수준 α 의 양측검정은,
- 귀무가설: $\mu = \mu_0$
- 대립가설: $\mu \neq \mu_0$

검정통계량으로 $Z = (\bar{X} - \mu_0) / \sqrt{\frac{\sigma^2}{n}}$ 을 사용하여

$$\begin{cases} Z < z_{1-\alpha/2} \text{ 또는 } z_{\alpha/2} < Z & \text{라면, 귀무가설을 기각} \\ z_{1-\alpha/2} \leq Z \leq z_{\alpha/2} & \text{라면, 귀무가설을 채택} \end{cases}$$

으로 수행한다



☐☐ ☐☐

2-1. ☐☐ - ☐☐☐☐☐ ☐☐☐ (☐☐☐ ☐ ☐)

.

[Python] 모분산을 알 때, 정규분포 모평균에 대한 검정

- $\mu_0 = 6$ 에 대해 유의수준 5%로 양측검정시,
p값이 0.055로서 0.05보다 크므로
귀무가설을 채택하게 됨

```
1 def pmean_test(sample, mean0, p_var, alpha=0.05):
2     s_mean = np.mean(sample)
3     n = len(sample)
4     rv = stats.norm()
5     interval = rv.interval(1-alpha)
6
7     z = (s_mean - mean0) / np.sqrt(p_var/n)
8     if interval[0] <= z <= interval[1]:
9         print('귀무가설을 채택')
10    else:
11        print('귀무가설을 기각')
12
13    if z < 0:
14        p = rv.cdf(z) * 2
15    else:
16        p = (1 - rv.cdf(z)) * 2
17    print(f'p값은 {p:.3f}')
```

```
1 pmean_test(sample, 6, 0.2)
```

귀무가설을 채택
p값은 0.055



가설 검정

2-2 검정 – 정규분포의 모분산



☒☒ ☒☒

2-2. 검정 – 정규분포의 모분산

.

정규분포 모분산에 대한 검정

- $X_1, X_2, \dots, X_n$ 이 서로 독립이고 $N(\mu, \sigma^2)$ 인 확률분포를 따르며
모분산 σ^2 에 관한 유의수준 α 의 양측검정은,
 - 귀무가설: $\sigma^2 = \sigma_0^2$
 - 대립가설: $\sigma^2 \neq \sigma_0^2$

검정통계량으로 $Y = \frac{(n-1)s^2}{\sigma_0^2}$ 을 사용하여

$$\left\{ \begin{array}{ll} Y < \chi_{1-\frac{\alpha}{2}}^2(n-1) \text{ 또는 } \chi_{\frac{\alpha}{2}}^2(n-1) < Y & \text{라면, 귀무가설을 기각} \\ \chi_{1-\frac{\alpha}{2}}^2(n-1) \leq Y \leq \chi_{\frac{\alpha}{2}}^2(n-1) & \text{라면, 귀무가설을 채택} \end{array} \right.$$

으로 수행한다



☐☐ ☐☐

2-2. ☐☐ - ☐☐☐☐☐ ☐☐☐

.

[Python] 정규분포 모분산에 대한 검정

- $\sigma_0^2 = 0.5$ 에 대해 유의수준 5%로 양측검정시,
p값이 0.047로서 0.05보다 작아
귀무가설을 기각하게 됨

```
1  def pvar_test(sample, var0, alpha=0.05):
2      u_var = np.var(sample, ddof=1)
3      n = len(sample)
4      rv = stats.chi2(df=n-1)
5      interval = rv.interval(1-alpha)
6
7      y = (n-1) * u_var / var0
8      if interval[0] <= y <= interval[1]:
9          print('귀무가설을 채택')
10     else:
11         print('귀무가설을 기각')
12
13     if y < rv.isf(0.5):
14         p = rv.cdf(y) * 2
15     else:
16         p = (1 - rv.cdf(y)) * 2
17     print(f'p값은 {p:.3f}')
```

```
1  pvar_test(sample, 0.5)
```

귀무가설을 기각
p값은 0.047



가설 검정

2-1 검정 – 정규분포의 모평균
(모분산 모를 때)



☒☒ ☒☒

2-3. 검정 – 정규분포의 모평균 (모분산 모를 때)

.

모분산을 모를 때, 정규분포 모평균에 대한 검정

모분산을 알지 못하는 상황에서 정규분포의 모평균에 대한 검정은 **1표본 t검정(1-sample t-test)**이라고 한다

- $X_1, X_2, \dots, X_n$ 이 서로 독립이고 $N(\mu, \sigma^2)$ 인 확률분포를 따르며

모평균 μ 에 관한 유의수준 α 의 양측검정은,

- 귀무가설: $\mu = \mu_0$

- 대립가설: $\mu \neq \mu_0$

검정통계량으로 $t = (\bar{X} - \mu_0) / \sqrt{\frac{s^2}{n}}$ 을 사용하여

$$\begin{cases} t < t_{1-\alpha/2}(n-1) \text{ 또는 } t_{\alpha/2}(n-1) < t & \text{라면, 귀무가설을 기각} \\ t_{1-\alpha/2}(n-1) \leq t \leq t_{\alpha/2}(n-1) & \text{라면, 귀무가설을 채택} \end{cases}$$

으로 수행한다



☐☐ ☐☐

2-3. ☐☐ - ☐☐☐☐☐ ☐☐☐ (☐☐☐ ☐☐ ☐)

.

[Python] 모분산 모를 때 정규분포 모평균에 대한 검정

- $\mu_0 = 6$ 에 대해 유의수준 5%로 양측검정시,
p값이 0.091로서 0.05보다 크므로
귀무가설을 채택하게 됨

```
1 def pmean_test(sample, mean0, alpha=0.05):
2     s_mean = np.mean(sample)
3     u_var = np.var(sample, ddof=1)
4     n = len(sample)
5     rv = stats.t(df=n-1)
6     interval = rv.interval(1-alpha)
7
8     t = (s_mean - mean0) / np.sqrt(u_var/n)
9     if interval[0] <= t <= interval[1]:
10         print('귀무가설을 채택')
11     else:
12         print('귀무가설을 기각')
13
14     if t < 0:
15         p = rv.cdf(t) * 2
16     else:
17         p = (1 - rv.cdf(t)) * 2
18     print(f'p값은 {p:.3f}')
```

```
1 pmean_test(sample, 6)
```

```
귀무가설을 채택
p값은 0.091
```



☐☐ ☐☐

2-3. ☐☐ - ☐☐☐☐☐ ☐☐☐ (☐☐☐ ☐☐ ☐)

.

[Python] 1표본 t검정 (1-sample t-test)

- Scipy.stats의 ttest_1samp 모듈을 활용하면 t 검정통계량과 p값을 얻을 수 있다

```
1 t, p = stats.ttest_1samp(sample, 6)
2 t, p

(1.781, 0.091)
```



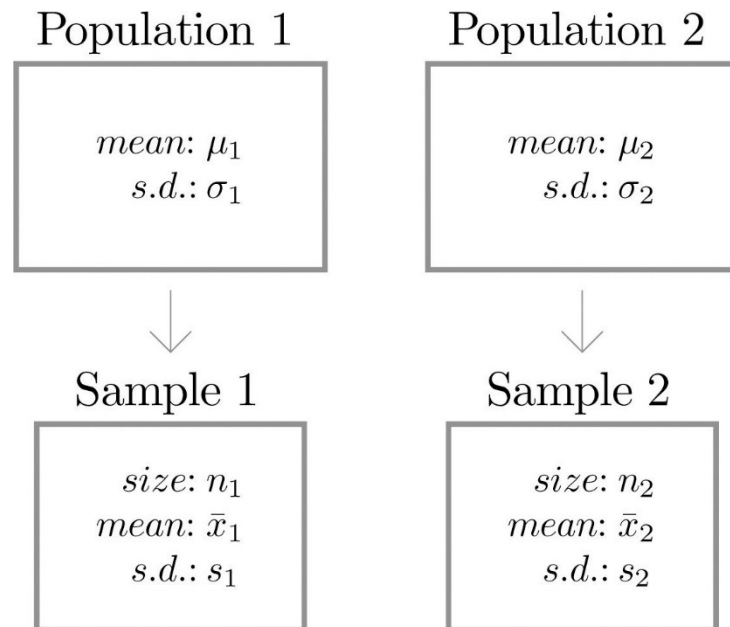
가설 검정

3-1 2표본 문제 – 대응비교 t 검정



2표본 문제 (two-sample problem)

- 2개 모집단에 관한 다양한 관계성 검정을 위한 문제로서,
2개 표본 각각의 대표값 사이에 차이가 있는지 검정을 통해 문제를 해결한다





2표본 문제 4가지 검정 방법

아래 두 가지 기준에 따라 네 가지로 분류된 검정 방법이 존재한다

1. 모집단에 정규분포를 가정할 수 있는지
2. 데이터가 서로 대응이 있는지 or 독립인지
 - 대응이 있다는 것은 2개 표본의 서로 대응되는 동일한 개체에 대해 다른 조건으로 측정한 것을 의미한다
(예시) 환자에게 약을 투여하기 전과 후의 활성화 수치를 비교하고자 할 때
 - 대응이 없다는 것은 개체가 서로 다른 독립 표본데이터로서 구성됨을 의미한다
(예시) 1반 학생들과 2반 학생들의 시험 점수를 비교하고자 할 때

	정규분포를 가정할 수 있음	정규분포를 가정할 수 없음 (간단히 살펴볼 예정)
대응표본	대응비교 t 검정	윌콕슨의 부호순위검정
독립표본	독립비교 t 검정	만·위트니의 U 검정



대응비교 t 검정 (Paired t-test)

- 대응하는 데이터가 존재하고, 데이터 차이에 정규분포를 가정할 수 있는 경우에 평균값 차이에 대한 검정 방법

대응하는 데이터 예시

- 근력운동을 진행하면 집중력이 향상되는지 여부를 파악하기위해,
20명의 사람을 선정하여 2주간 근력운동 하기 전/후의 집중력 테스트 결과를 생성하였다



가설 검정

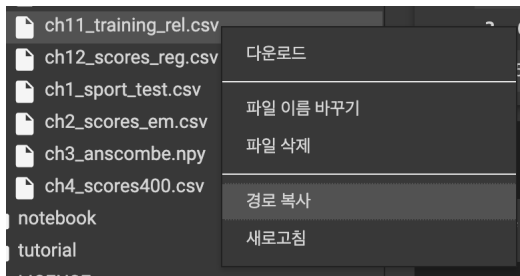
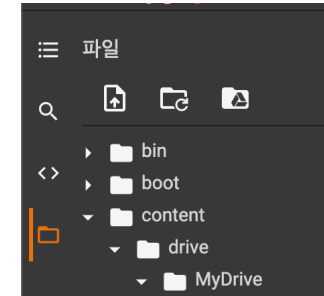
3-1. 2표본 문제 – 대응비교 t 검정

.

데이터 파일을 읽기위한 구글드라이브 활용 방법

1. 구글드라이브에 데이터 파일 업로드
2. 주어진 소스를 실행해서 구글드라이브 마운트 진행
3. Colab화면 좌측 상단의 4번째 '폴더' 이미지를 선택해서
content 폴더 아래에 drive/MyDrive 폴더로 접근해서 파일을 찾는다
4. 아래 그림처럼 파일에서 오른쪽버튼을 활용하여 경로 복사 후
pandas.read_csv로 데이터를 읽는다

```
1 from google.colab import drive
2 drive.mount('/content/drive')
```



```
1 # 주소복사 예시
2 training_rel = pd.read_csv('<복사된주소>')
```



[Python] 대응비교 t 검정

- 근력운동 전/후 집중력 테스트 결과값 데이터 파악

```
1 training_rel = pd.read_csv('/content/drive/MyDrive/@@2021 Fast Campus 강의/확률및통계/ch11_training_rel.csv')
2 print(training_rel.shape)
3 training_rel.head()'
```

(20, 2)

	전	후
0	59	41
1	52	63
2	55	68
3	61	59
4	59	84



가설 검정

3-1. 2표본 문제 – 대응비교 t 검정

.

대응비교 t 검정

- 근력운동 전/후 집중력 테스트 결과값 차이 비교 (양측 검정)
 - 귀무가설 : $\mu_{after} - \mu_{before} = 0$
 - 대립가설 : $\mu_{after} - \mu_{before} \neq 0$

[Python] 차이에 대한 변수 생성

```
1 training_rel['차'] = training_rel['후'] - training_rel['전']  
2 training_rel.head()
```

	전	후	차
0	59	41	-18
1	52	63	11
2	55	68	13
3	61	59	-2
4	59	84	25



대응비교 t 검정

- 근력운동 전/후 집중력 테스트 결과값 차이 비교 (양측 검정)
 - 귀무가설 : $\mu_{diff} = \mu_{after} - \mu_{before} = 0$
 - 대립가설 : $\mu_{diff} = \mu_{after} - \mu_{before} \neq 0$
 - * 차이값들의 평균이 0인지 아닌지 여부를 검정하는 것과 동일

[Python] 차이값 분포를 scipy.stats의 ttest_1samp로 1표본 t검정 계산하기

- P값이 0.04로서 0.05보다 작으므로 귀무가설은 기각되고,
근력운동이 집중력에 유의한 차이를 가져오는 것을 확인할 수 있다

```
1 t, p = stats.ttest_1samp(training_rel['차'], 0)
2 p
0.040
```



[Python] 차이값 분포를 `scipy.stats`의 `ttest_rel`로 전/후 데이터 기반 동일한 검정 계산

- 차이값 분포의 1표본 t검정 결과와 동일하게,
P값이 0.04로서 0.05보다 작으므로 귀무가설은 기각되고,
근력운동이 집중력에 유의한 차이를 가져오는 것을 확인할 수 있다

```
1 t, p = stats.ttest_rel(training_rel['후'], training_rel['전'])
2 p
0.040
```



가설 검정

3-2 2표본 문제 – 독립비교 t 검정



☒☒ ☒☒

3-2. 2표본 문제 – 독립비교 t 검정

독립비교 t 검정 (Independent t-test)

- 대응하는 데이터가 없고, 독립된 2표본 모집단에 정규분포를 가정할 수 있는 경우의 평균값 차이에 대한 검정 방법

독립비교 t 검정 데이터 예시

- A학급은 근력운동을 평소에 잘 하지 않는 집단이고,
B학급은 체육을 좋아해서 근력운동을 평소에 꾸준히 하는 집단이다
근력운동을 자주 하는 집단과 그렇지 않은 집단간 집중력 차이가 있는지 여부를 파악하기위
해,
독립적인 두 학급에서 집중력 테스트 결과를 생성하였다



[Python] 독립비교 t 검정

- A학급/B학급 집중력 테스트 결과값 데이터 파악

```
1 training_ind = pd.read_csv('/content/drive/MyDrive/@@2021 Fast Campus 강의/확률및통계/ch11_training_ind.csv')
2 print(training_ind.shape)
3 training_ind.head()
```

(20, 2)

	A	B
0	47	49
1	50	52
2	37	54
3	60	48
4	39	51



독립비교 t 검정

- A학급/B학급 집중력 테스트 결과값 차이 비교 (양측 검정)
 - 귀무가설 : $\mu_A - \mu_B = 0$
 - 대립가설 : $\mu_A - \mu_B \neq 0$
- 독립비교 t 검정의 특징
 - 대응이 없는 독립표본 데이터이므로 개별 데이터 간의 차이를 구하는 것은 의미가 없다



독립비교 t 검정

- A학급/B학급 집중력 테스트 결과값 차이 비교 (양측 검정)
 - 귀무가설 : $\mu_A - \mu_B = 0$
 - 대립가설 : $\mu_A - \mu_B \neq 0$
- A학급의 학급 점수 $X_1, X_2, \dots, X_{n_1}$ 은 $N(\mu_1, \sigma_1^2)$ 를 따르고,
B학급의 학급 점수 $Y_1, Y_2, \dots, Y_{n_2}$ 은 $N(\mu_2, \sigma_2^2)$ 를 따른다고 할 때

검정통계량은 $t = \frac{(\bar{X} - \bar{Y}) - (\mu_1 - \mu_2)}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$ 가 활용되며, 자유도는 $\frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{s_1^4}{n_1^2(n_1-1)} + \frac{s_2^4}{n_2^2(n_2-1)}}$ 인 t 분포를 따른다.

이를 '웰치의 방법'이라고 표현한다



- A학급/B학급 집중력 테스트 결과값 차이 비교 (양측 검정)
 - 귀무가설 : $\mu_A - \mu_B = 0$
 - 대립가설 : $\mu_A - \mu_B \neq 0$
- 수식이 매우 복잡해보이지만, stats.ttest_ind 함수를 활용하면 간단히 계산이 가능하며, 또한 equal_var=False를 지정하면 웰치의 방법으로 계산이 가능하다

=> p값이 유의수준이 0.05보다 크기 때문에, 귀무가설이 채택된다



윌콕슨의 부호순위검정 (Wilcoxon signed-rank test)

- 대응하는 데이터에서, 독립된 2표본 모집단에 정규분포를 가정할 수 없는 경우
중앙값 차이에 대한 검정 방법

윌콕슨의 부호순위검정 방법

1. 방법이름 그대로 대응하는 데이터간의 차이를 절대값을 기준으로 순위로 나타낸다
2. 차이의 부호가 마이너스인 것의 순위합 및 플러스인 것의 순위합을 고려하여 더 작은 쪽이 검정통계량이 되며, 임계값 기준보다 작으면 차이가 있다는 주장을 할 수 있다
3. Scipy.stats의 wilcoxon함수를 활용하면, 이와 같이 부호의 순위합을 계산하고, 표준화를 수행한 뒤 정규분포로 검정을 통해서 검정통계량 값이 약간 바뀌긴 하지만, 기본 원리에는 차이가 없다.



만.위트니의 U검정 (Mann-Whitney rank test)

- 대응되는 데이터가 없는, 독립된 2표본 모집단에 정규분포를 가정할 수 없는 경우 중앙값 차이에 대한 검정 방법으로서, 윌콕스의 순위합검정(Not 부호순위검정)이라고도 불린다
- 2표본 데이터 전체에 대해서 값이 작은 순서대로 순위를 부여하고, 검정통계량은 한 표본 데이터의 순위합을 사용한다
 - 좋은 순위가 모여있으면, 순위합이 작아지고 반대면 커지는 것을 통해 데이터 편향을 잘 반영할 수 있기 때문이다.
- U 검정의 검정통계량은 한 표본 데이터의 크기를 n_1 이라 할 때, 순위합에서 $\frac{n_1(n_1+1)}{2}$ 를 뺀 값이다
- Scipy.stats의 mannwhitneyu 함수로 사용해서 U검정을 할 수 있으며,



가설 검정

3-3 2표본 문제 – 카이제곱 검정



☒☒ ☒☒

3-3. 2표본 문제 – 카이제곱 검정

.

독립성 검정 (Test for Independence)

- 두 변수 X와 Y에 관해 ‘X와 Y가 독립이다’라는 귀무가설을 기반으로 수행되는 검정
- 독립성 검정에는 카이제곱분포가 사용되기 때문에 **카이제곱검정 (Chi-square Test)** 라고도 불리운다

[Python] 독립성 검정

- ‘광고’와 ‘상품 구입 유무’ 데이터 활용

```
1 ad_df = pd.read_csv(  
2     '/content/drive/MyDrive/@@2021 Fast Campus 강의/확률및통계/ch11_ad.csv'  
3 )  
4 n = len(ad_df)  
5 print(n)  
6 ad_df.head()
```

```
1000  
   광고  구입  
0    B  하지 않았다  
1    B  하지 않았다  
2    A    했다  
3    A    했다  
4    B  하지 않았다
```



[Python] 독립성 검정 방법

1. 교차집계표 (Cross Table) 작성 – 도수분포표의 2변수 버전

```
1 ad_cross = pd.crosstab(ad_df['광고'], ad_df['구입'])  
2 ad_cross
```

구입 하지 않았다 했다

광고

A	351	49
B	549	51



[Python] 독립성 검정 방법

2. A와 B 각각의 광고를 통해 상품을 구입한 비율이 어떻게 되는지 알아보기

```
1 ad_cross['했다'] / (ad_cross['했다'] + ad_cross['하지 않았다'])

광고
A      0.1225
B      0.0850
dtype: float64
```

3. 상품을 구입한 사람의 합계, 상품을 구입하지 않은 사람의 합계, 광고 A를 본 사람의 합계, 광고 B를 본 사람의 합계를 구한다

```
1 n_not, n_yes = ad_cross.sum()
2 n_not, n_yes

(900, 100)

1 n_adA, n_adB = ad_cross.sum(axis=1)
2 n_adA, n_adB

(400, 600)
```



[Python] 독립성 검정 방법

- 광고와 상관없이 상품을 구입한 비율은 10%이며,
광고와 구입이 독립이어서 광고에 따라 상품을 구입한 비율이 변하지 않는다면,
A 광고를 본 후 구입하게 될 기대도수는 $400\text{명} \times 10\% = 40\text{명}$ 이다
실제로 관측된 데이터는 관측도수라고 한다

4. 모든 셀에서 기대도수의 계산을 수행한다

```
1 ad_ef = pd.DataFrame({'했다': [n_adA * n_yes / n,  
2                           n_adB * n_yes / n],  
3                       '하지 않았다': [n_adA * n_not / n,  
4                                       n_adB * n_not / n]},  
5                           index=['A', 'B'])  
6 ad_ef
```

	했다	하지 않았다
A	40.0	360.0
B	60.0	540.0



[Python] 독립성 검정 방법

5. 관측도수와 기대도수의 i 번째 행, j 번째 열의 성분들을 각각 O_{ij} 와 E_{ij} 라 하면, 기대도수와 관측도수의 차이를 측정하여 아래의 검정통계량을 계산한다

$$Y = \sum_i \sum_j \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

이 Y 는 자유도가 1인 카이제곱분포를 근사적으로 따른다고 알려져 있어서 p 값을 간단히 구할

```
1 y = ((ad_cross - ad_ef) ** 2 / ad_ef).sum().sum()
2 y

3.750

1 rv = stats.chi2(1)
2 1 - rv.cdf(y)

0.053
```



[Python] 독립성 검정 방법

- Scipy.stats의 chi2_contingency 함수를 사용해서도 간단하게 계산이 가능하다
인수에는 교차집계표와 파라미터인 correction=False를 주면
반환값으로 검정통계량, 자유도, 기댓도수를 얻을 수 있다

```
1 chi2, p, dof, ef = stats.chi2_contingency(ad_cross,  
2 | | | | | | | | | | correction=False)  
3 chi2, p, dof  
  
(3.750, 0.053, 1)
```

Q&A

질의응답 (5분)





다음 주 배울 내용

Contents for Next Week

- Kaggle / Dacon 데이터 분석 사례 Case Study
- 회귀분석(확률 및 통계)

끝. 감사합니다.

수업 듣느라 수고하셨습니다.

